Alma Mater Studiorum · Università di Bologna

Scuola di Ingegneria e Architettura
Dipartimento di Informatica · Scienza e Ingegneria · DISI
Corso di Laurea Magistrale in Ingegneria Informatica

GoDP: una nuova soluzione per la gestione e la differential privacy dei dati sensibili

Relatore:
Prof. Michele Colajanni

Correlatore:
Ing. Silvio Russo

Presentata da:
Riccardo Barbieri

Anno Accademico 2023/2024

Elenco delle figure

2.1 Guadagno di informazione sul dataset al variare di epsilon 18
2.2 Distribuzione di Laplace 19
2.3 Distribuzioni di Laplace per dataset con e senza utente con multiple lamentele 21
2.5 Distribuzione differenzialmente privata con post-processing applicato 23
2.7 Distribuzione differenzialmente privata con soglia applicata (>= 2) . 24
2.8 Parametro δ al variare della soglia 25
2.11 Inversa normalizzata della PLRV 28
2.12 Distribuzioni di Laplace e normale a confronto, $\varepsilon = 1, \delta = 10^{-5}$ 29

4.1 Modello DP centralizzata 46
4.2 Fasi del difference attack: le sottrazioni successive isolano il contributo della vittima. 48
4.3 Confronto tra i risultati dell'attacco con e senza protezione DP (sinistra) e errore relativo al variare di ε (destra). 52
4.4 Rumore assoluto per query al variare di ε e confronto con il parametro di scala teorico $b = \Delta f/\varepsilon_{\text{agg}}$. 53

Indice

Elenco delle figure **3**

1 Stato dell'arte **9**

1.1 Introduzione alla Privacy Differenziale 9

1.2 Contesto storico e metodi tradizionali per la protezione della privacy . 10

1.3 Utilizzi DP 10

1.4 DP e machine learning 11

1.5 Strumenti per la DP 12

2 Meccanismi Differenzialmente Privati **15**

2.1 Definizione 15

2.1.1 Quantificazione della conoscenza dell'attaccante 16

2.2 Meccanismo di Laplace 18

2.2.1 Definizione 19

2.2.2 Meccanismo di Laplace e singole statistiche 20

2.2.3 Meccanismo di Laplace e statistiche multiple 21

2.3 (ε, δ)-privacy differenziale 23

2.3.1 Definizione 25

2.3.2 Perdita di privacy 26

2.3.3 Rumore gaussiano 28

3 GoDP - una CLI per la privacy differenziale **31**

3.1 Architettura 32

3.1.1 Panoramica componenti 32

3.1.2 Modulo aggregazioni 33

3.1.3 Modulo modello dei dati 36

3.1.4 Modulo composizione pipeline 38

3.1.5 Modulo budget 39

3.2 DPYaml - definizione di pipeline DP 41

4 Attacco di de-anonimizzazione **45**
4.1 Threat model 45
4.2 Difference attack 46
4.2.1 Dataset sperimentale 48
4.3 Configurazione dell'esperimento e risultati 49
4.3.1 Attacco senza protezione DP 50
4.3.2 Attacco con protezione DP 51
4.3.3 Analisi del rumore 52
4.4 Discussione 54
4.4.1 Efficacia della protezione 54
4.4.2 Trade-off privacy-utilità 54
4.4.3 Ruolo della selezione di partizione 54
4.4.4 Limiti dell'esperimento 55

Appendices **57**

A Derivazione equazioni **59**

B Dimostrazioni **61**

C Codice sorgente **63**

Bibliografia **65**

Introduzione

La privacy dei dati è diventata una preoccupazione sempre più importante nell'era digitale. Con la crescente raccolta e l'analisi di dati personali, è fondamentale garantire che le informazioni individuali siano protette da accessi non autorizzati e da usi impropri. La privacy differenziale (DP) è una rigorosa definizione matematica di privacy che offre forti garanzie di riservatezza anche quando i dati vengono utilizzati per l'analisi.

1 Stato dell'arte

1.1 Introduzione alla Privacy Differenziale

La privacy differenziale (DP) è un rigoroso framework matematico per la progettazione di algoritmi utilizzati per la creazione di distribuzioni di dati aggregati su dataset; gli algoritmi che rispettano la definizione di DP sono in grado di limitare l'impatto che la partecipazione di un singolo individuo ha sui risultati dell'analisi, aggiungendo una ragionevole quantità di rumore casuale, rendendo *quasi* impossibile confermare o negare la presenza di un soggetto all'interno del dataset utilizzato per l'aggregazione di dati [1]. Questa proprietà è cruciale per prevenire che eventuali attaccanti possano inferire informazioni sensibili su individui partecipanti al dataset, specialmente in presenza di fonti di informazione ausiliarie.

Il framework della DP nasce con l'obiettivo di sopperire alla necessità di pubblicare statistiche aggregate partendo da raccolte di dati sensibili limitando la probabilità di esporre informazioni private su singoli soggetti del database. In particolare, l'obiettivo dei meccanismi DP è quello di limitare la probabilità che sia possibile confermare o negare la presenza di un soggetto in un dataset osservando il risultato di una aggregazione sullo stesso [2].

Gli algoritmi DP sono specialmente utili grazie alla resistenza ad attacchi di deanonimizzazione che sfruttano informazioni da dati provenienti da altre fonti [3], caratteristica desiderabile in un contesto moderno che vede una quantità costantemente crescente di dati potenzialmente privati di singoli soggetti esposti al pubblico sia a causa di data breach [4] che a causa di diffusione intenzionale su social network e piattaforme simili.

Con la proliferazione di machine learning, big data e la generale condivisione pervasiva di dati, le tecniche di anonimizzazione tradizionali si sono rivelate inadeguate [2]. Queste tecniche falliscono nel garantire privacy ai soggetti di un dataset a causa della disponibilità di dati da fonti come social network [5], piattaforme di streaming [6] e dispositivi di geo-localizzazione [7], permettendo a un attaccante di associare dati sensibili agli individui.

1.2 Contesto storico e metodi tradizionali per la protezione della privacy

Il concetto di privacy differenziale è stato introdotto formalmente da Cynthia Dwork [8], dove viene formalizzata la definizione di ε-DP e dimostrato come l'aggiunta di rumore calibrato alla sensitività di una funzione possa proteggere la privacy individuale mantenendo un certo grado di utilità dei dati.

Prima dell'avvento della DP, l'anonimizzazione dei dati si basava su tecniche come k-anonimity, basata sulla soppressione o generalizzazione di determinati campi del dataset [9], e altre tecniche derivate come t-closeness e l-diversity che impongono limiti più stretti sulle distribuzioni dei valori degli attributi [10]. Queste tecniche permettono di creare distribuzioni di dati che, prese da sole, garantiscono un alto livello di privacy, richiedendo metodi sofisticati per estrarre informazioni utili; sono tuttavia più vulnerabili ad attacchi di rianonimizzazione che sfruttano informazioni ausiliarie [11].

Un gestore di dati che effettua pubblicazioni di statistiche su questi dati non è in grado di controllare quali informazioni aggiuntive sui partecipanti al dataset sono disponibili o verranno rese disponibili in futuro a un potenziale attaccante, la DP costituisce una soluzione ideale rispetto ai metodi tradizionali, data la particolare resistenza agli attacchi che si basano sulla disponibilità di informazioni aggiuntive.

Un'altra differenza fondamentale che la DP presenta rispetto a metodi tradizionali è la possibilità di quantificare il livello di privacy perso con una distribuzione di dati tramite un apposito parametro; questa caratteristica, supportata dalle caratteristiche di composizione della DP[12], permette di stabilire e tracciare un budget di privacy per uno specifico dataset al contrario dei metodi tradizionali sopra citati, combinando nel modo corretto i parametri di privacy.

1.3 Utilizzi DP

Il framework della DP è stato adottato in diversi ambiti, a partire dall'analisi lessicale per sistemi di suggerimenti per tastiere alla raccolta di informazioni sull'utilizzo di piattaforme web; di seguito verranno documentati alcuni esempi.

Apple utilizza la DP con modello locale per anonimizzare i dati raccolti da dispositivi iOS e macOS [13]; questi dati vengono utilizzati per migliorare le funzionalità di suggerimenti per tastiera e identificazione di siti web che causano interruzioni o altri errori nel browser Safari.

Google ha utilizzato la DP in svariati contesti, in particolare ha spesso utilizzato il framework per analisi dall'enorme mole di dati GPS ottenuti dai dispositivi con un

account Google. Una di queste analisi, Community Mobility Reports [14], nasce per quantificare cambiamenti in pattern di mobilità durante la pandemia COVID-19: qui la DP permette di non pubblicare mai valori assoluti sulle visite.

La Audience Engagements API di Linkedin costituisce un altro interessante esempio di applicazione del framework perché si tratta di un'implementazione del modello distribuito interattivo: diversamente dagli esempi precedenti, il rilascio dei dati non avviene tutto contemporaneamente, un *analista* richiede i dati di cui necessita in modo incrementale; questo sistema impone limitazioni nella quantità di dati accessibili in un dato periodo per poter garantire un certo livello di anonimizzazione [15].

1.4 DP e machine learning

Negli ultimi anni, modelli di machine learning vengono sfruttati sempre più spesso in ambiti che hanno a che fare con domini sensibili come sanità [16] e finanza [17], ambiti nei quali vengono spesso trattati dataset che includono informazioni riservate sugli individui.

Per questa tipologia di applicazione del machine learning sono attualmente in fase di sviluppo diversi metodi per limitare la perdita di privacy dei soggetti inclusi nei dataset utilizzati per l'addestramento [18] [19], ad esempio tecniche di protezione della privacy basate su crittografia omomorfa sono state implementate in sistemi di analisi di dati per la rilevazione di pattern che distinguono schemi di riciclaggio di denaro [20]. Tecniche come quest'ultima, oltre ad essere applicabili a una gamma di applicazioni ristretta data la natura estremamente specifica degli algoritmi coinvolti, comportano scelte basate su compromessi tra preservazione della privacy e efficienza computazionale.

L'allenamento di modelli ML su dati sensibili può portare, durante il loro utilizzo, alla distribuzione involontaria di questi dati [21]. Approfittando di tecniche come model inversion [22] e membership inference [23] potenziali attaccanti in grado di sfruttare le vulnerabilità connesse presentano un rischio significativo per la privacy degli individui e per la sicurezza delle organizzazioni che gestiscono tali dati.

Per sopperire a queste vulnerabilità, sono state sviluppate diverse tecniche che incorporano la DP durante l'addestramento o durante l'inferenza sui dati.

Uno dei primi esempi di questi metodi è la tecnica *Differentially Private Stochastic Gradient Descent* [24] che consiste in una variante dell'algoritmo SGD standard che prevede l'aggiunta di rumore accuratamente calibrato ai gradienti a ogni iterazione del processo di addestramento, questo approccio combinato con l'imposizione di un limite superiore ai valori dei gradienti permette di limitare la ; questa tecnica è usata per addestrare modelli di deep learning.

Un altro approccio a questo problema nell'ambito della classificazione è *Private Aggregation of Teacher Ensembles*, tecnica che addestra un modello *studente* utilizzando i dati forniti da un insieme di modelli *insegnanti* addestrati su set disgiunti del dataset iniziale; l'output fornito dall'insieme di insegnanti è reso privato con l'aggiunta di rumore opportunamente calibrato [25].

L'approccio di federated learning consente di allenare un modello di machine learning in maniera distribuita, senza dover condividere i dati utilizzati per l'addestramento; i client condividono il risultato dell'allenamento con un server centrale che agisce da aggregatore creando un unico modello.

Questa tecnica da sola garantisce un certo livello di privacy nei dati dato che non è necessario che l'aggregatore abbia accesso al dataset completo; le garanzie di privacy di questo modello possono tuttavia essere migliorate condividendo gli aggiornamenti dei modelli locali al server dopo averli mascherati con un algoritmo differenzialmente privato. Il rumore aggiunto assicura che la contribuzione di una singola voce del dataset non possa essere inferita facilmente [26].

1.5 Strumenti per la DP

Negli ultimi anni sono nati vari strumenti per permettere a utenti non tecnici di creare release di dati protette dalle garanzie offerte dalla DP.

Nell'ambito open source, i framework più utilizzati sono OpenDP e le Google Differential Privacy Libraries.

Uno degli strumenti più diffusi è *OpenDP*, una libreria open source sviluppata dall'Harvard University, che fornisce una suite di algoritmi per l'analisi statistica, ed è basata parzialmente sulle librerie di Google. Offre implementazioni in Rust e Python così come un'interfaccia SQL per integrare la libreria con workflow esistenti [27].

Le librerie di Google sono un insieme di strumenti per realizzare applicazioni che includono l'utilizzo della DP; Google fornisce una libreria di elementi base in Go, Java e C++ che include le primitive per realizzare aggregazioni differenzialmente private per l'aggiunta di rumore calibrato.
Questi elementi costituiscono la base del framework Privacy on Beam, un framework che integra tecniche di DP all'interno di pipeline di elaborazione dati composte con Apache Beam.
In questo pacchetto di Google sono inclusi anche componenti per la gestione del budget della privacy e per effettuare verifiche formali sulle garanzie offerte dalla DP.

Passando a strumenti enterprise, Tumult Labs propone una soluzione per applicazioni industriali ad alta scalabilità basata su Apache Spark distribuita sotto forma

di una libreria Python. Offrono soluzioni mirate al settore pubblico nel rispetto delle normative vigenti e al settore finanziario [28].

In questo lavoro di tesi si mira a creare uno strumento basato su Privacy on Beam con l'obiettivo di sviluppare una soluzione che consenta la creazione di release di dati differenzialmente private di piccoli-medi dataset tramite una semplice interfaccia da linea di comando e file di configurazione. Questo strumento è pensato per essere adatto a progetti di piccola entità, ad esempio per semplificare la creazione di release di dati per gruppi di ricerca, piccoli media outlet o interne a un'azienda.

2 Meccanismi Differenzialmente Privati

2.1 Definizione

La definizione seguente costituisce un formalismo più ristretto rispetto alla definizione completa, quest'ultima include un secondo parametro δ, la cui funzione e utilità verranno discusse in seguito.

Una trasformazione $\mathcal{M}$ è considerata ε-differenzialmente privata se, per ogni dataset D_1 e D_2 che differiscono al massimo per un individuo ($\|D_1 - D_2\|_1 \leq 1$) e ogni $S \subseteq \text{Range}(\mathcal{M})$, vale la seguente disequazione [8]:

$$\Pr[\mathcal{M}(D_1) = S] \leq e^{\varepsilon} \cdot \Pr[\mathcal{M}(D_2) = S] \tag{2.1}$$

I termini $\Pr[\mathcal{M}(D_1) = S]$ e $\Pr[\mathcal{M}(D_2) = S]$ indicano la probabilità che il risultato dell'applicazione del meccanismo $\mathcal{M}$ su D_1 e D_2 risulti nell'output $S \in \text{Range}(\mathcal{M})$. Il termine $e^{\varepsilon}, \varepsilon > 0$ rappresenta un coefficiente che indica quanto le due probabilità possono differire tra di loro, in particolare per valori di ε vicini a 0 le probabilità sono simili; all'aumentare di ε la differenza tra le due probabilità può aumentare.

Considerando che i due database D_1 e D_2 differiscono per un solo individuo, essi sono intercambiabili; ciò rende la definizione simmetrica. Vale quindi anche:

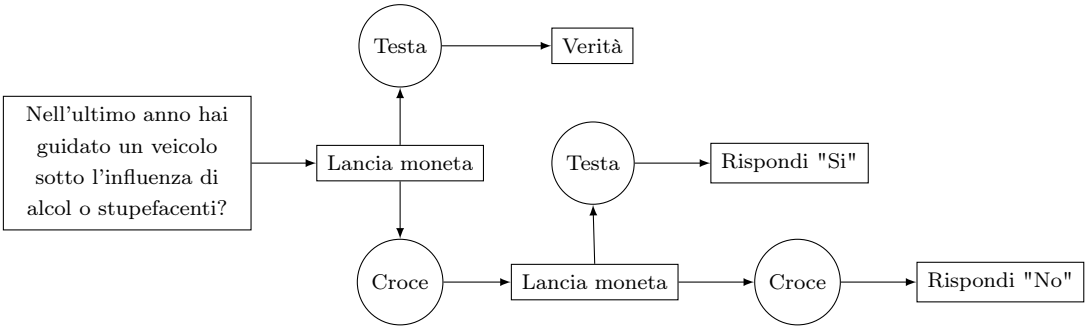
$$e^{-\varepsilon} \cdot \Pr[\mathcal{M}(D_2) = S] \leq \Pr[\mathcal{M}(D_1) = S] \leq e^{\varepsilon} \cdot \Pr[\mathcal{M}(D_2) = S] \tag{2.2}$$

Un esempio pratico

Si supponga di volere condurre un sondaggio su una popolazione ponendo la domanda: "Nell'ultimo anno hai guidato un veicolo sotto l'influenza di alcol o stupefacenti?". Ovviamente, una persona alla quale viene posta direttamente questa domanda tenderà a mentire allo scopo di proteggere la propria privacy e evitare di auto incriminarsi. In questa istanza può rendersi utile l'applicazione di un meccanismo differenzialmente privato.

Per ovviare a questa limitazione del processo di raccolta dati, si richiede ai partecipanti di lanciare una moneta senza mostrare il risultato:

- se il risultato è testa il soggetto risponde alla domanda in modo sincero
- se il risultato è croce il soggetto lancia una seconda moneta: su testa risponde "Si", su croce "No"



Questo processo introduce un elemento casuale alla risposta di un singolo soggetto, offrendo all'individuo una negazione plausibile contro eventuali accuse e impedendo a terze parti di discriminare sulla base delle risposte [29].

Nel complesso, riscrivendo i risultati nei termini della definizione si ottiene:

- $\Pr[\mathcal{M}(Si) = Si] = 0.75$, $\Pr[\mathcal{M}(Si) = No] = 0.25$
- $\Pr[\mathcal{M}(No) = Si] = 0.25$, $\Pr[\mathcal{M}(No) = No] = 0.75$

Considerati questi risultati si può affermare che il meccanismo illustrato è $\ln 3$-differenzialmente privato; si può dimostrare partendo dalla definizione (2.1) che:

$$\frac{\Pr[\mathcal{M}(D_1) = S]}{\Pr[\mathcal{M}(D_2) = S]} \leq e^{\varepsilon}$$
$$\frac{\Pr[\mathcal{M}(Si) = Si]}{\Pr[\mathcal{M}(Si) = No]} \leq 3 = \frac{\Pr[\mathcal{M}(No) = No]}{\Pr[\mathcal{M}(No) = Si]}$$

Il meccanismo illustrato dimostra in termini semplici come la privacy differenziale possa agire sui dati per proteggere la privacy dei partecipanti che aderiscono a essere inseriti in una raccolta di dati. Come specificato in precedenza, questo processo garantisce negazione plausibile sul contenuto della risposta ai soggetti partecipanti; non fornisce tuttavia la possibilità di negare di aver partecipato al sondaggio. Al contrario, la privacy differenziale è applicabile solamente a distribuzioni di dati basate su sistemi di interrogazione che generano risultati aggregati.

2.1.1 Quantificazione della conoscenza dell'attaccante

Il parametro ε presente nella definizione di ε-DP può essere interpretato come una misura del livello massimo di conoscenza che un potenziale attaccante può guadagnare osservando i risultati di un'interrogazione del meccanismo differenzialmente privato.

Prendendo come riferimento un attaccante che conosce l'intero database eccetto il soggetto target si può modellare il suo punto di vista considerando due possibili dataset: D_{in} contiene il target mentre D_{out} no; l'obiettivo dell'attaccante è capire quale dei due database è quello reale.

Nel modello dell'attaccante si può esprimere il sospetto che il database D sia D_{in} come $\Pr[D = D_{in}] = 1 - \Pr[D = D_{out}]$. Si supponga ora che l'attaccante abbia osservato che il meccanismo DP applicato a D restituisce l'output O: si può esprimere la probabilità che $D = D_{in}$ dopo aver osservato l'output come $\Pr[D = D_{in}|\mathcal{M}(D) = O]$; applicando il teorema di Bayes si ottiene un'espressione che permette di quantificare il livello di sospetto $Pr[D = D_{in}|M(D) = O]$ che l'attaccante ha dopo aver osservato l'output O.

$$\Pr[D = D_{in}|\mathcal{M}(D) = O] = \frac{\Pr[D = D_{in}] \cdot \Pr[\mathcal{M}(D) = O|D = D_{in}]}{\Pr[\mathcal{M}(D) = O]} \tag{2.3}$$

Il termine $\Pr[D = D_{in}]$ è il sospetto iniziale e il termine $\Pr[\mathcal{M}(D) = O|D = D_{in}]$ può essere riscritto come $\Pr[\mathcal{M}(D_{in}) = O]$ indicando la probabilità che il meccanismo $\mathcal{M}$ applicato a D_{in} restituisca O. Mentre i primi due termini sono conosciuti, il termine $\Pr[\mathcal{M}(D) = O]$ è sconosciuto; per rimediare si può considerare il rapporto tra i due sospetti aggiornati all'output O.

$$\frac{\Pr[D = D_{in}|\mathcal{M}(D) = O]}{\Pr[D = D_{out}|\mathcal{M}(D) = O]} = \frac{\Pr[D = D_{in}]}{\Pr[D = D_{out}]} \cdot \frac{\Pr[\mathcal{M}(D_{in}) = O]}{\Pr[\mathcal{M}(D_{out}) = O]} \tag{2.4}$$

In questa espressione emerge il termine $\frac{\Pr[\mathcal{M}(D_{in})=O]}{\Pr[\mathcal{M}(D_{out})=O]}$, la definizione di ε-DP (2.1) limita il rapporto come segue:

$$e^{-\varepsilon} \leq \frac{\Pr[\mathcal{M}(D_{in}) = O]}{\Pr[\mathcal{M}(D_{out}) = O]} \leq e^{\varepsilon} \tag{2.5}$$

Applicando all'equazione (2.4) si ottiene:

$$e^{-\varepsilon} \cdot \frac{\Pr[D = D_{in}]}{\Pr[D = D_{out}]} \leq \frac{\Pr[D = D_{in}|\mathcal{M}(D) = O]}{\Pr[D = D_{out}|\mathcal{M}(D) = O]} \leq e^{\varepsilon} \cdot \frac{\Pr[D = D_{in}]}{\Pr[D = D_{out}]} \tag{2.6}$$

Applicando le sostituzioni $\Pr[D = D_{out}] = 1 - \Pr[D = D_{in}]$ e $\Pr[D = D_{out}|\mathcal{M}(D) = O] = 1 - \Pr[D = D_{in}|\mathcal{M}(D) = O]$ e riorganizzando i termini si ottiene:

$$\frac{\Pr[D = D_{in}]}{e^{\varepsilon} + (1 - e^{\varepsilon}) \cdot \Pr[D = D_{in}]} \leq \Pr[D = D_{in}|\mathcal{M}(D) = O] \leq \frac{e^{\varepsilon} \cdot \Pr[D = D_{in}]}{1 + (e^{\varepsilon} - 1) \cdot \Pr[D = D_{in}]} \tag{2.7}$$

Per i passaggi della trasformazione, consultare l'appendice A.

Tracciando il grafico dell'espressione risultante al variare di ε si può notare come il guadagno di informazione potenziale possa aumentare drasticamente per valori di

ε alti.

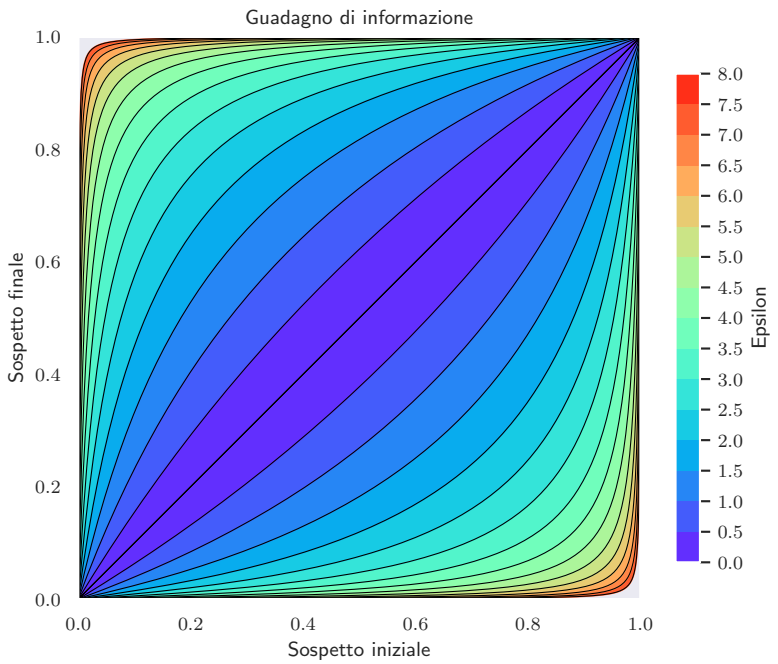


Figura 2.1: Guadagno di informazione sul dataset al variare di epsilon

2.2 Meccanismo di Laplace

Le query più comuni che vengono effettuate su database sono di tipo numerico, precisamente sono funzioni del tipo $f\colon \mathbb{N}^{|\mathcal{X}|} \to \mathbb{R}^k$, dove $\mathcal{X}$ rappresenta l'universo di possibili record in un database x.

Per rendere questa tipologia di query differenzialmente privata con il meccanismo di Laplace si aggiunge rumore campionato da una distribuzione di Laplace:

$$Lap(x|\mu, b) = \frac{1}{2b} \exp\left(-\frac{|x-\mu|}{b}\right) \tag{2.8}$$

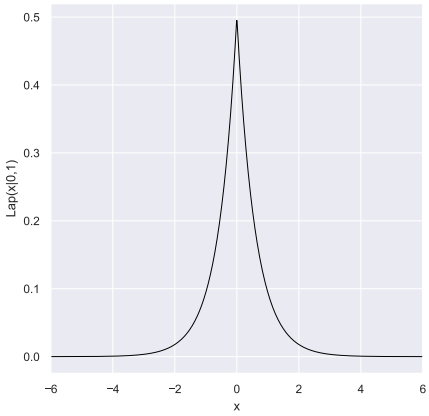


Figura 2.2: Distribuzione di Laplace

2.2.1 Definizione

Si consideri una qualsiasi funzione $f\colon \mathbb{N}^{|\mathcal{X}|} \to \mathbb{R}^k$, il meccanismo di Laplace è definito:

$$\mathcal{M}_L(x, f(\cdot), \varepsilon) = f(x) + (Y_1, \ldots, Y_k) \tag{2.9}$$

dove i parametri Y_i sono valori campionati da una distribuzione di Laplace. Per illustrare la calibrazione della distribuzione utilizzata è necessario introdurre alcuni concetti.

Distanza tra database

Si considerino due database $x, y \in \mathbb{R}^{|\mathcal{X}|}$. Per definire la distanza tra i due database si fa uso della norma ℓ_1, dove la norma ℓ_1 di un database è definita come:

$$||x||_1 = \sum_{i=1}^{|\mathcal{X}|} |x_i| \tag{2.10}$$

La distanza ℓ_1 tra due database x e y è $||x - y||_1$.

In questo caso $||x||_1$ è una misura del numero di record contenuti nel database e $||x - y||_1$ rappresenta quanti record sono diversi tra x e y.

Sensitività di una funzione

La sensitività di una funzione è un valore che indica la magnitudine massima che un cambiamento dei dati di un singolo individuo può causare sul risultato della funzione considerata; questo valore verrà utilizzato come parametro per calibrare la quantità di rumore che il meccanismo aggiungerà ai risultati delle query.

La sensitività ℓ_1 è definita come:

$$\Delta f = \max_{\substack{x,y \in \mathbb{N}^{|\mathcal{X}|} \\ ||x-y||_1 = 1}} ||f(x) - f(y)||_1 \tag{2.11}$$

Utilizzando la sensitività della funzione in considerazione si calibra la scala (parametro b) della distribuzione di Laplace come

$$Lap(x|0, \Delta f/\varepsilon) \tag{2.12}$$

Per la dimostrazione che l'utilizzo di questa distribuzione crea un meccanismo che rispetta ε-privacy differenziale consultare B

2.2.2 Meccanismo di Laplace e singole statistiche

Si supponga di avere a disposizione un dataset che raccoglie le lamentele poste dai clienti di un'azienda tramite un modulo di feedback, e di voler creare un report sul numero di lamentele poste al giorno; si decide di applicare il meccanismo di Laplace che aggiunge rumore calibrato per anonimizzare i risultati dell'analisi.

Il primo passo per creare un meccanismo di Laplace differenzialmente privato è calcolare la sensitività della funzione in considerazione. Dato che si tratta di un'operazione di conteggio, intuitivamente si può pensare che questo valore sia pari a 1; tuttavia è necessario considerare la *contribuzione massima* che un singolo utente può apportare al conteggio per valut-are correttamente casi in cui un individuo presenta più di una lamentela al giorno.

Se si considerasse $\Delta f = 1$ e quindi un parametro di scala per la distribuzione di Laplace pari a $b = \frac{1}{\varepsilon}$ si otterrebbe un meccanismo $x\varepsilon$-differenzialmente privato, dove x corrisponde alla massima contribuzione che un singolo individuo può apportare al dataset. Supponendo un dataset di 500 unità con un utente che ha posto 5 lamentele, utilizzando $\Delta f = 1$ si otterrebbero due distribuzioni di probabilità con rapporto massimo di $e^{5\varepsilon}$, violando quindi la definizione di privacy differenziale.

Si riporta di seguito il grafico che mostra le due distribuzioni di probabilità centrate sul numero di soggetti nei due database, uno con il soggetto con le 5 lamentele e l'altro senza, e il loro rapporto.

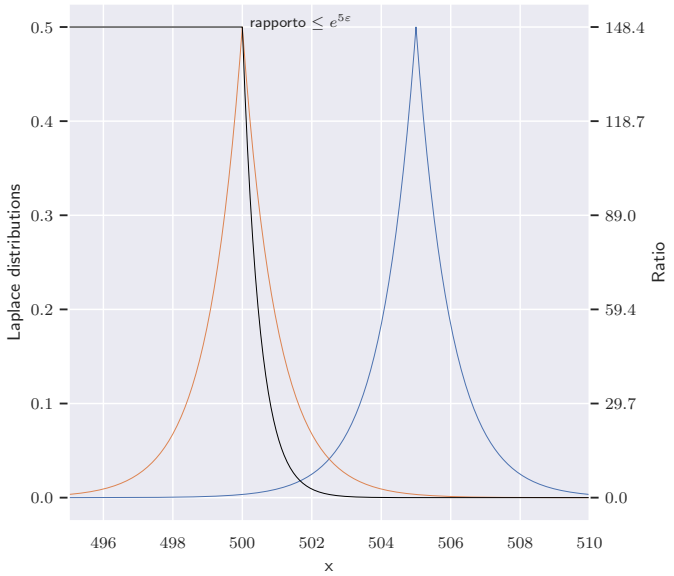


Figura 2.3: Distribuzioni di Laplace per dataset con e senza utente con multiple lamentele

Per rispettare i requisiti della privacy differenziale è necessario utilizzare la sensitività della funzione di conteggio tenendo conto della possibilità di avere multipli record legati a un singolo individuo.

Limitazione del contributo

Generalizzando, il problema ricade sulle caratteristiche del particolare dominio di applicazione; si rende quindi necessario l'intervento di un esperto del dominio allo scopo di definire un ragionevole limite massimo alla sensitività; in fase di analisi sarà necessario limitare il conteggio dei record relativi a un singolo individuo.

2.2.3 Meccanismo di Laplace e statistiche multiple

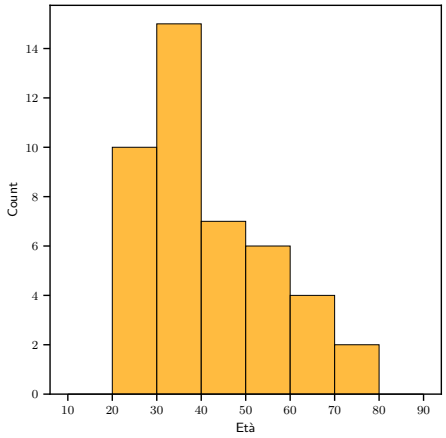
Nel caso in cui si vogliano rilasciare statistiche multiple su un dataset (e.g. istogrammi) è necessario considerare come viene applicato il rumore a ognuna delle statistiche o categorie calcolate per assicurarsi di non violare la definizione di privacy differenziale.

Si distinguono due casistiche differenti: un caso è caratterizzato da utenti che possono influenzare al massimo una statistica, che è quindi in questo senso disgiunta dalle altre, e un caso dove gli utenti possono influenzare diverse statistiche.

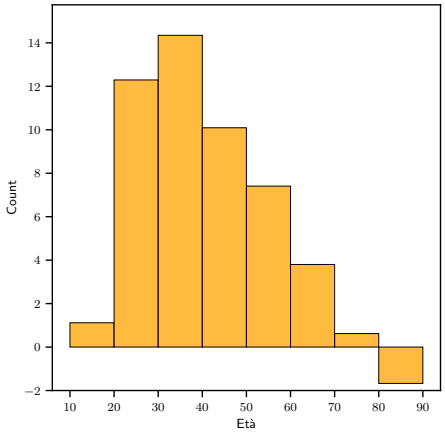
Caso 1: statistiche disgiunte

Quando le statistiche multiple da includere nella distribuzione di dati sono disgiunte, è sufficiente aggiungere rumore estratto da una distribuzione di Laplace di scala $\frac{\Delta f}{\varepsilon}$

a ogni statistica calcolata. Un esempio di questa eventualità è la creazione di un istogramma che mostra la distribuzione di età in un dataset.



(a) Distribuzione età soggetti



(b) Distribuzione differenzialmente privata

Il grafico di sinistra rappresenta i dati senza nessuna strategia di anonimizzazione, si crea il secondo istogramma utilizzando un meccanismo di Laplace con scala $\frac{1}{\varepsilon}$ che aggiunge rumore a ogni categoria utilizzando lo stesso parametro di scala. Come si può notare dai due grafici, ci sono evidenti problemi con la categoria 80-89 in quanto risulta negativa, così come con le altre categorie perché presentano valori non interi; in questo caso l'applicazione del meccanismo di Laplace ha dato luogo a valori incongruenti.

Per fare fronte a casi come questo in cui l'applicazione del meccanismo differenzialmente privato scelto porta i dati al di fuori del loro dominio (e.g. valori negativi per un conteggio, valori non interi) è lecito applicare funzioni di post-processing.

Considerando un meccanismo ε-differenzialmente privato $\mathcal{M}\colon \mathbb{N}^{|\mathcal{X}|} \to \mathbb{R}$ e un mapping casuale arbitrario $f\colon \mathbb{R} \to \mathbb{R}'$ allora $f \circ \mathcal{M}\colon \mathbb{N}^{|\mathcal{X}|} \to \mathbb{R}'$ è anch'esso ε-differenzialmente privato. Questo risultato è dimostrato formalmente in [3], tuttavia intuitivamente si può capire che una qualsiasi funzione di post-processing che applica una trasformazione ai dati non agisce sui dati del dataset originario, non può quindi aumentare la perdita di privacy che la privacy differenziale concede con il meccanismo utilizzato.

Applicando all'esempio di prima gli opportuni arrotondamenti e rimpiazzando i valori negativi con 0 si ottiene nuovamente una distribuzione di dati ε-differenzialmente privata.

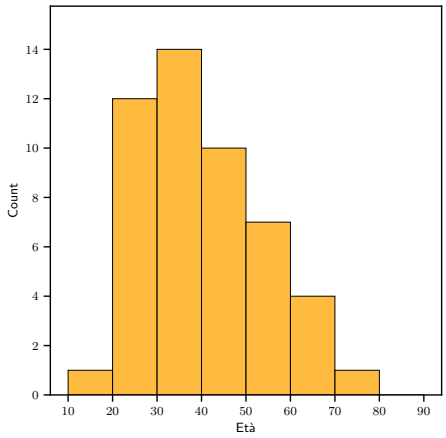


Figura 2.5: Distribuzione differenzialmente privata con post-processing applicato

Caso 2: statistiche intersecanti

Se le diverse statistiche da rilasciare possono essere influenzate dallo stesso utente, dato un numero C di statistiche sarà necessario calcolare la sensitività della funzione considerando quanto un singolo utente possa contribuire a tutte le statistiche, nel caso di conteggi o calcolo di medie sarà necessario considerare $\Delta f = C$ e un parametro di scala $b = C/\varepsilon$, in questo modo ogni statistica sarà ε/C-differenzialmente privata.

Questa strategia porta a evidenziare il vero significato del parametro ε come un valore che quantifica il **budget di privacy** assegnato a una particolare distribuzione di dati.

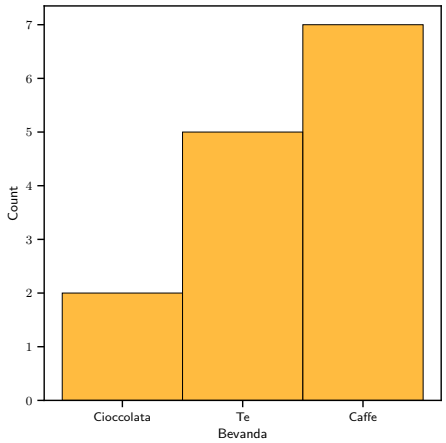
Una soluzione più flessibile rispetto a quella di creare C statistiche ε/C-DP è quella di suddividere il budget in parti diseguali $\varepsilon_1, \varepsilon_2, \ldots, \varepsilon_C$ tale che $\sum_{i=1}^{C} \varepsilon_i = \varepsilon$. Se esistono statistiche che sono più sensibili al rumore o necessitano di essere più accurate, è possibile allocare una porzione di budget maggiore: questo risulterà nell'aggiunta di una quantità di rumore minore, tuttavia sarà necessario aggiungere più rumore alle altre statistiche.

2.3 (ε, δ)-privacy differenziale

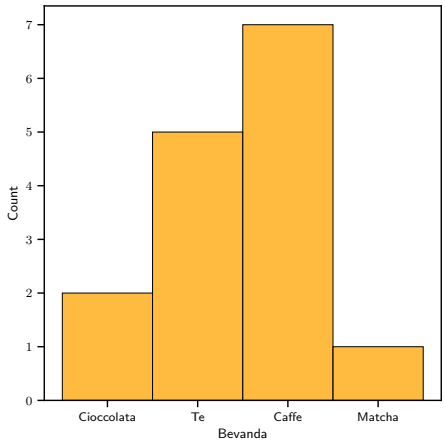
Si supponga di procedere con un sondaggio che pone domande ai soggetti senza un numero predefinito di possibili risposte e che può quindi dare vita a un numero di categorie non prestabilito.

Questo tipo di raccolta di dati è intrinsecamente soggetto a distorsioni, imprecisioni e, in generale, rumore nei dati raccolti, come ad esempio risposte di soggetti che fraintendono la domanda, risposte intenzionalmente non valide o semplicemente risposte rare.

Il problema che sorge in questi casi consiste nella possibilità di avere due database che differiscono per un solo soggetto, ma uno dei due perde una categoria rara.



(a) Distribuzione bevande database 1



(b) Distribuzione bevande database 2

Un attaccante che osserva questi risultati è in grado di capire con totale certezza quale dei due database contiene il soggetto di interesse: questo è un **evento distintivo**.

Una possibile soluzione consiste nell'applicare una soglia minima: si aggiunge rumore a tutte le categorie che sono state generate dalla raccolta di dati per poi scartare quelle che ricadono sotto a questa soglia per evitare *eventi distintivi* che possono infrangere la definizione di DP.

Nell'esempio di prima si potrebbe scegliere una soglia conservativa pari a 2, ottenendo come risultato il seguente istogramma.

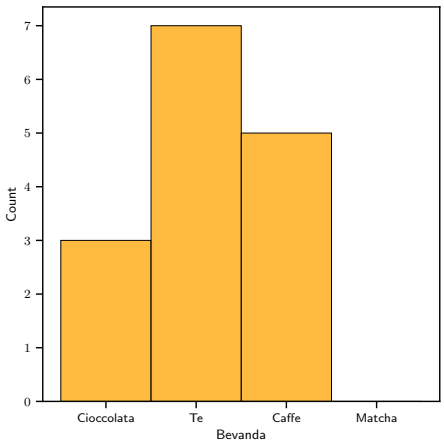


Figura 2.7: Distribuzione differenzialmente privata con soglia applicata (>= 2)

Questa soluzione non è perfetta, come si può vedere, la categoria rara "Matcha" viene rimossa dalla distribuzione di dati finale; la categoria "Cioccolata" tuttavia, pur ricadendo sotto la soglia nel dataset originario, è stata pubblicata in quanto

l'aggiunta di rumore dal meccanismo DP risulta sopra la soglia, questo caso viola la definizione di ε-DP.

2.3.1 Definizione

Per poter modellare una versione di DP che sia in grado di comprendere release di dati più vicine a casi realistici è necessario introdurre una nuova definizione: (ε, δ)-privacy differenziale, definizione che rilassa lo standard imposto dalla definizione di (ε)-DP introducendo il parametro $\boldsymbol{\delta}$. Questo termine può essere interpretato come la probabilità di pubblicare un evento distintivo.

$$\Pr[\mathcal{M}(D_1) \in S] \leq e^{\varepsilon} \cdot \Pr[\mathcal{M}(D_2) \in S] + \boldsymbol{\delta} \tag{2.13}$$

Nell'esempio di prima si fa utilizzo di un meccanismo (ε, δ)-DP con rumore campionato da una distribuzione di Laplace con una soglia pari a 2. Per stabilire quale parametro δ è stato utilizzato si calcola la probabilità che una categoria con conteggio 1, quindi esistente, superi la soglia utilizzata all'aggiunta di rumore di Laplace.

A questo scopo si fa uso della *funzione di sopravvivenza* della variabile aleatoria da cui si campiona il rumore per il meccanismo; data una variabile aleatoria X con funzione cumulativa di distribuzione $F(x)$, la funzione di sopravvivenza è definita come $S(x) = 1 - F(x) = P[X > x]$.

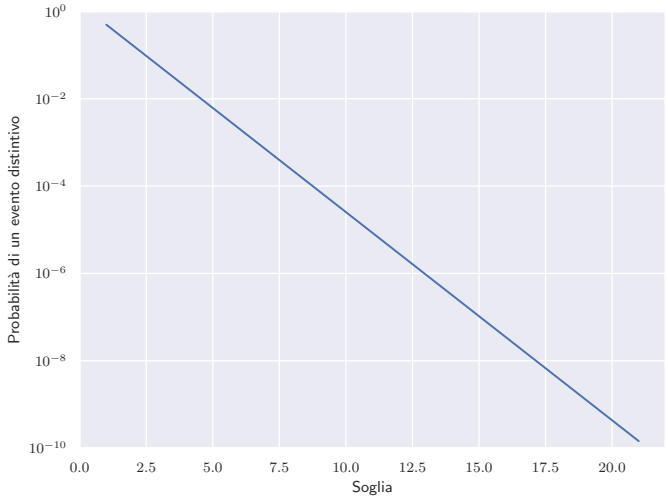


Figura 2.8: Parametro δ al variare della soglia

Dato che il parametro δ rappresenta la probabilità che un singolo utente ha di avere i propri dati esposti a seguito di una query al meccanismo DP, viene spesso scelto $\delta \ll 1/n$ in quanto la probabilità che i dati di *almeno un soggetto* vengano esposti è pari a $n\delta$: è necessario assicurarsi che questo valore sia abbastanza piccolo. Questa regola assume che i singoli eventi distintivi siano non correlati; spesso

questo fatto non è in realtà corretto, ma è un'ottima approssimazione in pratica. Ad esempio, in un database con 1 milione di utenti, basterebbe una soglia di 15 per ottenere un valore di $n\delta < 0.1$.

2.3.2 Perdita di privacy

Riprendendo la definizione di guadagno di informazione 2.4 con un meccanismo $\mathcal{M}$ definito ε-DP si può definire la seguente quantità.

$$\frac{\Pr[\mathcal{M}(D_{in}) = O]}{\Pr[\mathcal{M}(D_{out}) = O]} = \frac{\frac{\Pr[D=D_{in}|\mathcal{M}(D)=O]}{\Pr[D=D_{out}|\mathcal{M}(D)=O]}}{\frac{\Pr[D=D_{in}]}{\Pr[D=D_{out}]}} \tag{2.14}$$

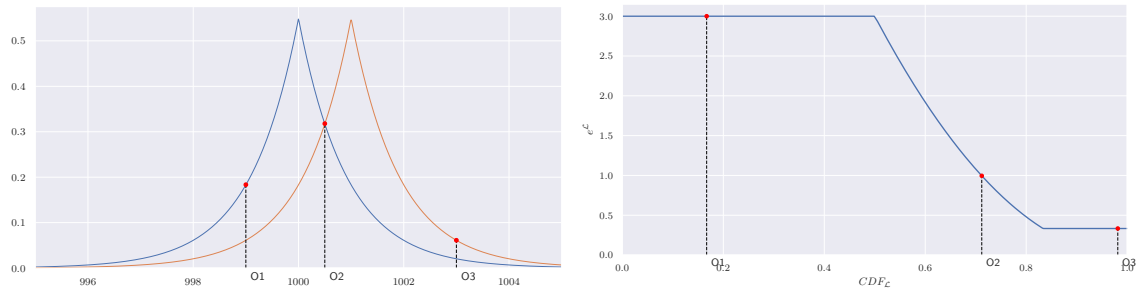
Combinando questa definizione bayesiana del guadagno di informazione dopo l'osservazione dell'output O con la definizione 2.1 che impone $\frac{\Pr[\mathcal{M}(D_{in})=O]}{\Pr[\mathcal{M}(D_{out})=O]} \leq e^{\varepsilon}$ si ottiene:

$$\mathcal{L}_{D_{in},D_{out}}(O) = \ln\left(\frac{\Pr[\mathcal{M}(D_{in}) = O]}{\Pr[\mathcal{M}(D_{out}) = O]}\right) \tag{2.15}$$

Questa quantità è definita come *variabile aleatoria della perdita di privacy* (PLRV) al variare di O e rappresenta il vero valore effettivo che ε assume per un dato output O.

Prendendo un parametro $\varepsilon = \ln(3)$ e utilizzando una distribuzione di Laplace, si traccia il grafico di $e^{\mathcal{L}}$ ponendo sulle ascisse gli eventi a seconda della loro probabilità, evidenziando come la perdita di privacy non superi mai e^{ε}.

Si includono nel grafico anche 3 possibili output arbitrari considerando un dataset di 1000 individui.



(a) Distribuzioni di Laplace per database D_{in} e D_{out}

(b) Quantità di perdita di privacy al variare della probabilità degli eventi

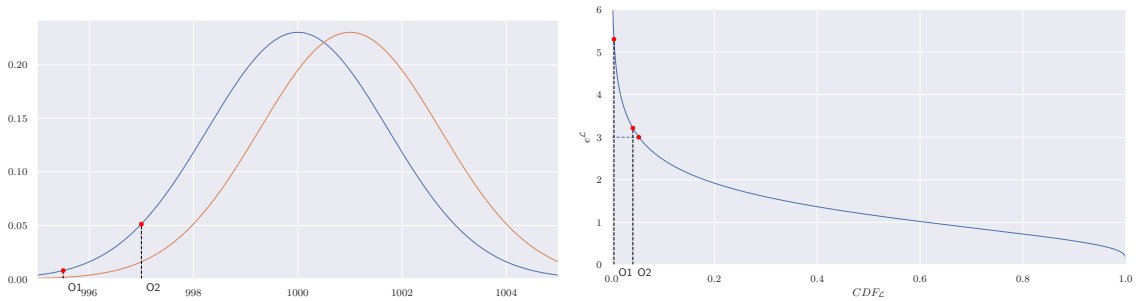
Il grafico mostra chiaramente come la conoscenza di un possibile attaccante cambia a seconda di quale output viene osservato:

- $\boldsymbol{O_1}$: è più probabile che questo output possa provenire da D_{in} che D_{out}, di preciso esattamente 3 volte più probabile come mostrato nel grafico 2.9b.
- $\boldsymbol{O_2}$: è equamente probabile che l'output possa provenire da D_{in} o da D_{out}.

- $\boldsymbol{O_3}$: è più probabile che l'output possa provenire dal database D_{out}, in questo caso la conoscenza dell'attaccante è divisa per 3 in quanto il vero database è D_{in}.

Allo scopo di evidenziare e visualizzare il preciso significato del parametro δ nella definizione di (ε, δ)-DP, si introduce un nuovo meccanismo DP che fa uso di rumore campionato da una *distribuzione di Gauss*; in seguito si motiverà l'esistenza di tale meccanismo DP nel contesto del rilascio di multiple statistiche.

Riprendendo l'esempio precedente ma utilizzando distribuzioni gaussiane come base del meccanismo si ottengono i due grafici seguenti:



(a) Distribuzioni gaussiane per database D_{in} e D_{out}

(b) Quantità di perdita di privacy al variare della probabilità degli eventi

Come si può vedere dal grafico di destra, con rumore gaussiano non esiste un limite superiore alla possibile perdita di privacy; la probabilità che si verifichi un evento distintivo è la probabilità che l'output restituito abbia un valore $e^{\mathcal{L}(O)} > 3$ e in questo caso è $\delta_1 \approx 0.053$, quindi il meccanismo è $(\ln(3), \delta_1)$-DP.

Pur essendo una caratterizzazione precisa del parametro δ, questo metodo non dà il giusto peso agli eventi molto più significativi degli altri; alcuni eventi, quelli *vicini* a δ_1, hanno un effetto molto minore sulla perdita di privacy con un aumento esponenziale man mano che l'evento si avvicina a una probabilità cumulativa pari a 0.

Per tenere conto di questa caratteristica degli eventi quasi distintivi si rielabora la curva della PLRV prendendone l'inverso per posizionare gli eventi con maggiore perdita di privacy vicino a $y = 0$, si normalizza poi la curva prendendo il rapporto $\frac{e^{\varepsilon}}{e^{\mathcal{L}}}$ per portare gli eventi non estremi vicino a $y = 1$.

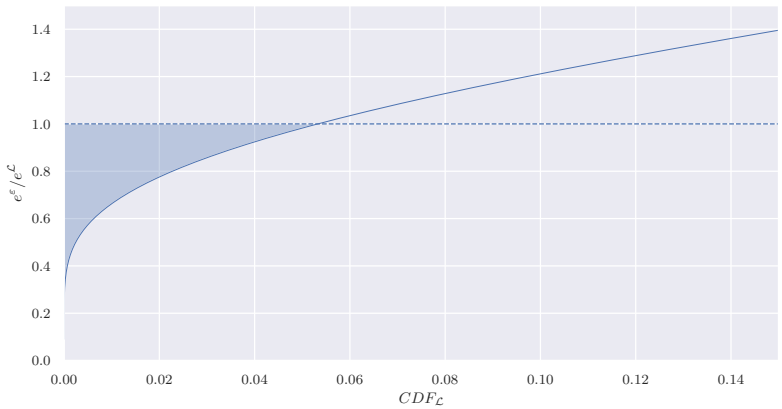


Figura 2.11: Inversa normalizzata della PLRV

Questa trasformazione della curva della PLRV identifica l'effettivo valore di δ come l'area sottesa tra la retta $y = 1$, ovvero il limite che separa eventi quasi distintivi da rilasci di dati che rispettano ε-DP, e la curva $e^{\varepsilon}/e^{\mathcal{L}}$ evidenziata nel grafico.

Per una dimostrazione che questa caratterizzazione identifica il valore di δ pesato sulla probabilità che $A(D_{in}) = O$ per ogni output O consultare l'appendice B.

2.3.3 Rumore gaussiano

Nella sezione precedente è stato utilizzato un meccanismo DP fondato sulla distribuzione normale per spiegare il significato del parametro δ nella definizione di (ε, δ)-DP, di seguito si discuterà per quale motivo è desiderabile creare meccanismi di questo tipo anche se ricadono in una categoria di meccanismi DP con garanzie minori rispetto a quelle della ε-privacy differenziale.

La distribuzione gaussiana ha proprietà convenienti, ad esempio può essere usata per modellare variabili aleatorie con distribuzione sconosciuta in quanto una somma di molte variabili aleatorie porta a una distribuzione normale (teorema del limite centrale); un altro vantaggio è la regola delle tre sigma [30] che afferma che una variabile aleatoria campionata da questa distribuzione sarà:

- nell'intervallo $[-\sigma, \sigma]$ 68% delle volte;
- nell'intervallo $[-2\sigma, 2\sigma]$ 95% delle volte;
- nell'intervallo $[-3\sigma, 3\sigma]$ 99.7% delle volte.

Confrontando questa distribuzione con la distribuzione di Laplace si nota che le code della funzione di distribuzione diminuiscono molto più velocemente nella prima; prendendo 1000000 di campioni dalle due distribuzioni, questi ricadranno nell'intervallo $[-5\sigma, 5\sigma]$ neanche 1% delle volte nella distribuzione di Laplace e il 99.99...% (12 volte 9) delle volte in una distribuzione normale.

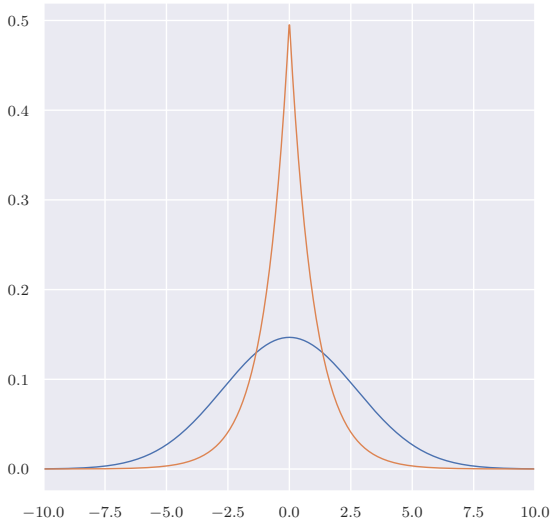


Figura 2.12: Distribuzioni di Laplace e normale a confronto, $\varepsilon = 1, \delta = 10^{-5}$

Mentre il meccanismo di Laplace è particolarmente adatto al rilasci di un numero limitato di statistiche, poiché garantisce ε-DP, il meccanismo gaussiano risulta più appropriato quando si necessita di rilasciare un numero elevato di statistiche, in quanto soddisfa una forma rilassata di privacy differenziale, ovvero (ε, δ)-DP, e presenta migliori proprietà di composizione.

Tuttavia, per garantire lo stesso livello di protezione della privacy, il meccanismo gaussiano richiede in genere l'aggiunta di una quantità di rumore maggiore rispetto al meccanismo di Laplace. In dataset di dimensione contenute tale rumore può avere un impatto significativo sul risultato dell'aggregazione, riducendo sensibilmente l'utilità dei dati.

3 GoDP - una CLI per la privacy differenziale

In questo capitolo si discuterà la creazione di `GoDP`, un'applicazione per la generazione di distribuzioni di dati protette dalle garanzie della DP. L'obiettivo principale è sviluppare uno strumento basato su Privacy on Beam che consenta la creazione di release di dati differenzialmente private di piccoli-medi dataset tramite una semplice interfaccia da linea di comando e file di configurazione. Questo strumento è pensato per essere adatto a progetti di piccola entità, come ad esempio la semplificazione della creazione di release di dati per gruppi di ricerca, piccoli media outlet o uso interno a un'azienda con l'obiettivo di ridurre la barriera di accesso a questa tecnica di anonimizzazione e separare un utente potenzialmente non esperto da dettagli implementativi.

L'applicazione sfrutta la libreria Privacy on Beam sviluppata da Google [31] e basata sul framework Apache Beam; la scelta di questa libreria è motivata dal fatto che Privacy on Beam è tra le implementazioni open source di algoritmi DP più diffuse e perché mette a disposizione una vasta gamma di primitive DP.

Si è scelto di utilizzare un framework DP basato su Apache Beam per l'implementazione di `GoDP` per il repertorio di caratteristiche che garantiscono flessibilità per compiti di elaborazione dati.

Tra queste caratteristiche figura l'unificazione del modello di programmazione tra batch processing e stream processing con apposite primitive per la definizione delle *pipeline* di elaborazione dati, ammettendo una netta separazione tra business logic e modello di elaborazione dei dati. Un'altra caratteristica che rende questo framework ideale per un'applicazione di questo tipo è la portabilità delle implementazioni, questa caratteristica è ottenuta grazie alla possibilità di eseguire una pipeline su una varietà di *runner* il che permette di distribuire un'applicazione in un qualsiasi ambiente che supporti l'implementazione di un runner, e.g. Direct Runner per testing e sviluppo, GC Dataflow Runner per GCP e Apache Flink Runner per altre piattaforme cloud. Questa caratteristica rende triviale la possibilità di migrare tra ambienti riducendo la possibilità di vendor lock-in per deployment su cloud.

3.1 Architettura

In questa sezione si documenterà l'architettura dell'applicazione `GoDP`, analizzando i componenti che la costituiscono e come questi interagiscono per generare distribuzioni di dati differenzialmente private.

L'architettura dell'applicazione permette due approcci per definire le aggregazioni differenzialmente private da elaborare:

- file di configurazione con apposito formato di definizione `DPYaml` con il quale si specificano le operazioni da compiere
- definizione di funzioni personalizzate per la realizzazione di operazioni complesse

La seconda metodologia è stata progettata per contemplare casistiche dove si vuole effettuare specifiche operazioni sul dataset che il formato di definizione `DPYaml` non contempla; questo approccio offre un'interfaccia di utilizzo relativamente semplice per utenti inesperti, lasciando tuttavia spazio per l'estensione dell'applicazione per operazioni complesse a utenti che hanno esperienza con il framework e la privacy differenziale.

3.1.1 Panoramica componenti

`GoDP` è composto da 4 moduli principali:

- `aggregations`
- `model`
- `runs`
- `budget`

In aggiunta a questi moduli, l'applicazione fa uso di altri componenti minori: il componente `cleaning` contiene funzioni per pulire i dati del dataset in input, come ad esempio la capitalizzazione di stringhe per uniformare eventuali valori malformati; il componente `io`, in combinazione con `format`, mette a disposizione funzioni utili alla scrittura e formattazione dei dataset generati dalle pipeline di elaborazione dati sfruttando la riflessione per poter gestire una vasta gamma di tipi di dato. Questi moduli minori, in particolare `io`, espongono metodi che vengono utilizzati per caricare i dataset da elaborare che, nella versione più recente dell'applicazione, devono essere formattati come file CSV.

Un altro componente integrale al funzionamento dell'applicazione è `global_-env`, questo componente dichiara funzioni e variabili globali utili alla condivisione di risorse come il riferimento alla pipeline e oggetti connessi all'elaborazione corrente, predispone inoltre la funzione centrale all'inizializzazione dei parametri globali dell'applicazione. Altre risorse sono state dichiarate come globali oltre a quelle in

`global_env`, questa strategia si è resa necessaria a causa di alcuni specifici dettagli implementativi del framework e del linguaggio utilizzato.

Un ulteriore modulo di complessità minore è `commands`, questo componente sfrutta la libreria Go `spf13/cobra` per creare la struttura dei comandi dell'interfaccia da linea di comando, definendone i metodi di utilizzo e i parametri supportati e richiesti. Le inizializzazioni di apposite strutture dati associano a ogni comando la funzione che deve essere lanciata quando il comando viene invocato, così come stringhe che ne descrivono l'utilizzo e forniscono esempi. Le funzioni in questione sono parte del modulo `runs` che verrà descritto in seguito.

3.1.2 Modulo aggregazioni

Il modulo `aggregations` è il componente principale dell'applicazione, al suo interno sono contenute le implementazioni delle trasformazioni differenzialmente private che `GoDP` supporta. In questo modulo sono implementate sia le operazioni differenzialmente private *generiche* sia quelle *specializzate*; di seguito si discuterà la differenza tra le due.

Per operazione specializzata si intende una trasformazione che non è possibile specificare tramite il formato di configurazione `DPYaml`; un esempio di operazione di questo tipo è una qualsiasi trasformazione che richiede di utilizzare valori composti o post-processati da campi del dataset.

Per descrivere al meglio il funzionamento delle funzioni di elaborazione dati, è necessario introdurre alcuni concetti alla base di Apache Beam:

- `Pipeline`: centro dell'operatività di Apache Beam, una pipeline è un grafo che contiene le operazioni definite dal programmatore;
- `PCollection`: contiene i dati da processare nella pipeline, può essere un dataset finito oppure uno stream di dati che viene rifornito in modo continuo;
- `Scope`: entità utilizzata per associare una operazione a una particolare pipeline, può essere usata per generare sotto-scope utili a raggruppare logicamente le trasformazioni e
- `ParDo`: una delle trasformazioni offerte dal framework, agisce applicando una funzione user-defined a ogni elemento di una `PCollection`, similarmente alla fase Map di un algoritmo Map/Shuffle/Reduce.

Le funzioni che implementano le strategie di elaborazione DP condividono la stessa firma di chiamata, questa è composta da:

- `scope` della pipeline, utilizzato per aggiungere le trasformazioni alla pipeline corrente;
- `pcol beam.PCollection`, contiene i dati da elaborare;

- `op model.OperationType`, incapsula i parametri dell'operazione da svolgere e
- `bd godp.DpBudget`, struttura dati che contiene il budget DP della pipeline ed espone metodi per ottenere il budget per la specifica operazione.

Queste funzioni condividono anche i passi che compiono per registrare le aggregazioni nella pipeline:

1. creazione di un sotto-scopo per da utilizzare per l'operazione
2. creazione di una `PrivatePCollection` (ppcol) a partire dalla `PCollection` che contiene i dati da elaborare
3. applicazione di una funzione a ogni elemento della `ppcol` per estrarre i dati di interesse, si genera una nuova `PrivatePCollection`
4. applicazione della trasformazione DP configurata con i parametri appropriati

Le `PCollection` supportano l'applicazione di trasformazioni arbitrarie ai dati che contengono, tuttavia per applicare trasformazioni differenzialmente private a questa struttura dati è necessario trasformarla in una `PrivatePCollection`, una struttura dati esposta da Privacy on Beam che associa ogni elemento di una `PCollection` a un *identificatore di privacy*. La scelta dell'identificatore determina le *unità di privacy* che saranno sotto la garanzia di (ε, δ)-DP, in particolare i risultati delle aggregazioni effettuate su una determinata `PrivatePCollection` saranno (ε, δ)-indistinguibili dal risultato ottenuto dall'applicazione delle aggregazioni sulla stessa `PrivatePCollection` dalla quale sono stati rimossi tutti i record associati a un determinato identificatore.

Per creare una `PrivatePCollection` Privacy on Beam espone metodi appositi da utilizzare in funzione del tipo di dato contenuto nella `PCollection`, quello utilizzato da questa applicazione è il metodo `pbeam.MakePrivateFromStruct` che accetta in input una `PCollection` di struct e il nome del campo dello struct che contiene l'identificatore di privacy. `GoDP` è stato progettato per poter accomodare dataset arbitrari, questo tuttavia si è rivelato un problema in fase di progettazione; inizialmente l'approccio utilizzato è stato quello di sfruttare la riflessione di Golang per creare a runtime un'entità struct dinamica che contenesse i campi presenti del dataset CSV, applicando questa metodologia Privacy on Beam è emerso che il framework non supporta tipi generici nella generazione di `PrivatePCollection`, la motivazione per questa limitazione risiede nel fatto che il framework non è in grado di inferire adeguatamente struct di dati arbitrariamente complessi a fronte della necessità di serializzare questa struttura.

Per far fronte a questo problema si è scelto di utilizzare un tipo di dato struct custom `ValuesStruct`, la scelta della struttura verrà discussa in seguito nella sezione sul modulo `model`.

Questo approccio rende necessario l'inclusione di una sezione del formato di

configurazione `DPYaml` dedicata a specificare, per i campi rilevanti, il tipo di dato contenuto nel dataset; questo dettaglio verrà discusso nella sezione dedicata al formato YAML.

Una volta ottenuta la `PrivatePCollection` si applica una funzione che estrae i campi da aggregare a tutti i valori della raccolta di dati, creando una seconda `PrivatePCollection` che contiene un sottoinsieme del dataset e si passa come parametro della funzione di aggregazione DP appropriata. L'applicazione di questa funzione richiede l'utilizzo di specifici parametri DP; oltre ai parametri δ e ε è necessario definire valori che sono propri del dataset da analizzare come ad esempio `MaxValue`, un valore che indica il valore massimo che un singolo soggetto può contribuire all'aggregazione considerata (2.2.2); questi parametri di configurazione verranno discussi in dettaglio nella sezione su `DPYaml`.

Si riporta di seguito un estratto del sorgente che evidenzia le caratteristiche descritte, questa è la funzione che implementa la funzione di aggregazione DP di conteggio degli elementi di un campo del dataset.

```
func CountColumn(scope beam.Scope, col beam.PCollection, op model.OperationType,
↪  bd healthcaredp.DpBudget) (*beam.PCollection, error) {
        scope = scope.Scope(op.OperationName)
        pCol := pbeam.MakePrivateFromStruct(scope, col, bd.PrivacySpec, "Id")

        pColumnValues := pbeam.ParDo(scope,
                func(struc model.ValuesStruct) string {
                        return struc.Values[op.Column]
                }, pCol)
        pColumnValuesCount := pbeam.Count(scope, pColumnValues,
            pbeam.CountParams{
                PartitionSelectionParams: pbeam.PartitionSelectionParams{
                    Epsilon:
↪  bd.GetBudgetShare(op.OperationName).PartitionEpsilon,
                    Delta:   bd.GetBudgetShare(op.OperationName).PartitionDelta,
                },
                AggregationEpsilon:
↪  bd.GetBudgetShare(op.OperationName).AggregationEpsilon,
                AggregationDelta:
↪  bd.GetBudgetShare(op.OperationName).AggregationDelta,
                MaxPartitionsContributed:
↪  *op.PrivacyParams.MaxCategoriesContributed,
                MaxValue: *op.PrivacyParams.MaxContributions,
            })
        return &pColumnValuesCount, nil
}
```

3.1.3 Modulo modello dei dati

Questo modulo è responsabile di definire le entità coinvolte nel flusso di dati dell'applicazione, così come un sotto-modulo di utilità dedicato a incapsulare la complessità del caricamento, formattazione e pulizia dei dati.

Le entità principali in questione sono due:

- `YamlConfig` e
- `ValueStruct`

Come intuibile dai nomi delle strutture dati, la prima è destinata a contenere le informazioni relative al file YAML specificato dall'utente per la configurazione dell'esecuzione, mentre la seconda adempie al compito di immagazzinare i valori del dataset da elaborare.

Del caricamento della configurazione YAML si occupa il file `go yaml_config` che opera leggendo il file specificato da linea di comando e effettua il parsing della struttura YAML. La libreria `goccy/go-yaml` fornisce un modo semplice per associare una serie di tipi struct innestati ai tag YAML corrispondenti. Utilizzando questa libreria in combinazione `go-playground/validator` è stato possibile definire appropriate regole di validazione dei valori di configurazione, la libreria associa poi appositi messaggi di errore ben formattati per indicare con precisione all'utente la posizione del campo mal specificato e il motivo dell'errore. Data la carenza di espressività di queste regole, che permettono di intervenire soltanto a livello del singolo campo, è stata definita una funzione che effettua controlli trasversali tra i campi, in particolare viene validato che per certi valori di un campo siano popolati specifici campi e assenti altri.

Oltre alla definizione dei tipi di dato, il modulo espone una singola funzione che richiede in input il nome di un file e restituisce lo struct di tipo `YamlConfig` opportunamente compilato.

Si riporta di seguito un estratto rilevante della definizione dei tipi di dato in questione.

```
type PrivacyBudgetType struct {
        NoiseKind        string
↪  `yaml:"noise_kind" validate:"required,oneof=gauss laplace"`
        Delta            float64 `yaml:"delta" validate:"required"`
        Epsilon          float64 `yaml:"epsilon" validate:"required"`
        AggregationShare float64
↪  `yaml:"aggregation_share" validate:"required,gt=0,lt=1"`
}

type PipelineDp struct {
        Configuration ConfigurationType
↪  `yaml:"configuration" validate:"required"`
```

```
        PrivacyBudget PrivacyBudgetType
↪  `yaml:"privacy_budget" validate:"required"`
        Types         []TypeType        `yaml:"types" validate:"required"`
        Operations    []OperationType   `yaml:"operations" validate:"required"`
}

type YamlConfig struct {
        PipelineDp PipelineDp `yaml:"pipelinedp" validate:"required"`
}
```

Come si può vedere dall'estratto, per ogni campo degli struct definiti viene specificato un *tag* rilevabile da librerie che usano la riflessione per aggiungere funzionalità; in questo caso si associa ogni elemento a un campo nella definizione YAML con l'omonimo tag e si specifica una serie di regole di validazione con il tag `validate`.

Un'altra sezione di questo modulo definisce la struttura dati `ValuesStruct`, riportata di seguito, insieme a metodi e variabili globali indispensabili per interagire con i dati all'interno degli struct.

```
type ValuesStruct struct {
        Values map[string]string
        Id     string
}
```

Questa struttura dati è pensata per immagazzinare un record del dataset in una mappa che utilizza i nomi dei campi come chiavi nel membro `Values` e il valore dell'identificatore di privacy nel membro `Id`; questo approccio, combinato con la possibilità di definire funzioni anonime, permette di gestire un dataset arbitrario senza conoscerne i campi a priori.

Le variabili globali citate in precedenza sono definite come segue:

- `var Headers []string`: contiene i nomi dei campi del dataset, viene usato per effettuare alcuni controlli di consistenza tra i dati di configurazione inseriti dall'utente e gli effettivi nomi dei campi;
- `var TypesMap map[string]string`: questa mappa viene composta a partire dalla configurazione fornita, ha come chiave il nome di un campo e come valore una stringa che ne indica il tipo e
- `var IdFieldIndex int`: memorizza l'indice che indica la posizione della colonna scelta come identificatore di privacy dall'utente nel'array `Headers`.

Un esempio ideale per motivare la necessità di queste variabili globali è la funzione seguente, della quale si riporta solo la firma per brevità.

```
func CreateGenericStruct(line string, emit func(struc ValuesStruct)) error
```

La funzione `CreateGenericStruct`, usata in combinazione con l'operazione `ParDo` di Apache Beam, *emette* uno per volta gli elementi che comporranno la `PCollection` a partire dalla lettura delle righe del dataset una alla volta; la particolare firma della funzione è richiesta dalla funzione `ParDo` del framework Apache Beam, che applica la funzione a ogni singola riga del dataset separatamente, rendendo impossibile fornire un contesto alla funzione che crea lo struct per una data riga.

3.1.4 Modulo composizione pipeline

Il modulo `runs` è responsabile di mettere insieme e orchestrare l'esecuzione degli altri componenti dell'applicazione; le funzioni definite in questo modulo vengono invocate direttamente dal framework `spf13/cobra` utilizzato per la creazione dell'interfaccia da linea di comando.

Come per le funzioni nel modulo `aggregations`, anche in `runs` figurano due tipologie diverse di funzioni di esecuzione della pipeline: quelle specializzate su un particolare dataset con rispettivo struct definito e quella generica che si basa unicamente sul contenuto del file di configurazione fornito.

Le funzioni specializzate sono quelle funzioni *run* che creano una pipeline a partire da un dataset CSV, una lista di operazioni predisposte e scrivono sul file di output prendendo i valori necessari alla configurazione direttamente dai parametri da linea di comando, è inoltre fondamentale che le operazioni siano state definite in precedenza dall'utente. Di seguito si riportano le azioni che queste funzioni eseguono:

1. `InitEnvironment`: funzione invocata prima di ogni run specializzata; qui viene configurata la pipeline, aggiunta alla a quest'ultima l'operazione di caricamento del dataset e raccolti i valori dei parametri da CLI;
2. inizializza il budget DP a seconda della operazioni richieste creando un oggetto `DpBudget` condiviso;
3. aggiunge alla pipeline le operazioni necessarie per eseguire le aggregazioni specificate chiamando le apposite funzioni dal modulo `aggregations`
4. esegue la pipeline con l'appropriato `Runner`

Non si entra nel dettaglio dell'implementazione delle funzioni specializzate in quanto eseguono semplicemente l'orchestrazione delle operazioni implementate da altri moduli aggiungendo solamente operazioni minori (e.g. gestione dei nomi dei file di output).

La funzione `RunFromFile` incapsula invece l'orchestrazione nel caso di utilizzo tramite file di configurazione; dato che i parametri CLI gestiti sono diversi da quelli

del caso di esecuzioni specializzate, non viene utilizzata `InitEnvironment` ma una logica diversa implementata direttamente all'interno della *run*.

Quando la funzione viene invocata inizia con l'ottenere il parametro CLI che specifica il file di configurazione per poi usarlo per leggere il file di configurazione e caricarlo nelle apposite strutture dati tramite la funzione `model.LoadYamlConfig`, successivamente utilizza questa configurazione per creare l'oggetto `DpBudget` per la pipeline corrente, questi due oggetti verranno discussi in dettaglio nelle sezioni successive. La funzione passa poi a interagire con il file che contiene il dataset, estraendo i nomi dei campi e scrivendo una versione temporanea del file con la prima riga rimossa, questa operazione è resa necessaria dal fatto che non è possibile differenziare una riga contenente dati dalla riga contenente i nomi dei campi, non sarebbe quindi possibile caricare correttamente i dati negli struct generici.

Successivamente la funzione aggiunge alla pipeline l'operazione che carica i dati dal file contenente i dati da analizzare, per poi aggiungere alla pipeline le aggregazioni specificate nel file di configurazione in ordine di dichiarazione, così come le operazioni di scrittura e stampa delle `PCollection` risultanti.

Infine, dopo aver compilato la pipeline nella sua interezza, la funzione invoca il `DirectRunner` messo a disposizione da Apache Beam che esegue tutte le operazioni associate alla pipeline in precedenza. Questa è la sezione del programma che è necessario modificare nel caso in cui si voglia eseguire la pipeline in un contesto diverso con un Runner specifico per particolari ambienti (e.g. piattaforme cloud).

3.1.5 Modulo budget

Il modulo `budget` è il componente che implementa le strutture dati e le operazioni per gestire il budget DP dedicato a una pipeline. Per questo scopo, il modulo dichiara le due strutture dati riportate di seguito.

```go
type DpBudget struct {
        PrivacySpec  *pbeam.PrivacySpec
        BudgetShares map[string]DpBudgetShare
        Delta        float64
        Epsilon      float64
}

type DpBudgetShare struct {
        AggregationEpsilon float64
        PartitionEpsilon   float64
        AggregationDelta   float64
        PartitionDelta     float64
}
```

Lo struct `DpBudget` definisce i campi `Delta` e `Epsilon` per mantenere i valori del budget di privacy totale per la pipeline per riferimento in altri metodi.

Oltre a questi due valori viene anche memorizzata la struttura dati `pbeam.PrivacySpec`, inizializzata dalle funzioni apposite definite in questo modulo, si tratta dello struct che i metodi del framework Privacy on Beam richiedono come argomento a ogni invocazione di una aggregazione differenzialmente privata che *consuma* il budget DP.

Nella struttura `DpBudget` è presente anche una mappa con chiave stringa e valore `DpBudgetShare`; questa struttura dati definita sopra immagazzina le quattro porzioni dei parametri di privacy che sono state dedicate all'operazione il cui nome è usato come chiave nella mappa. La definizione della strategia con cui viene allocato il budget è parte del formato di definizione `DPYaml` e verrà discussa in seguito.

Una delle funzioni utilizzate per inizializzare il budget è definita come metodo associato allo struct `DpBudget` come *ricevitore*, questa è una caratteristica di Golang che permette di invocare metodi da struct con una forma di dot-notation; questi metodi hanno accesso al contenuto della struttura dati sulla quale sono stati invocati e, se sono dichiarati su un *ricevitore puntatore*, possono anche modificarne il contenuto. Si riporta di seguito come esempio la firma della funzione che inizializza il budget a partire dalla definizione YAML.

```go
func (db *DpBudget) InitYamlBudgetShares(config *model.YamlConfig) (err error)
```

Questa funzione agisce compilando i valori della struttura dati a partire dalla configurazione passata come parametro, differenziando tra caso con rumore "laplaciano" e rumore "gaussiano" per creare la `pbeam.PrivacySpec` e le singole quote del budget.

La struttura dati `DpBudgetShare` può risultare in apparenza ridondante in quanto duplica i dati già disponibili nella `pbeam.PrivacySpec` all'interno di una struttura personalizzata; si tratta tuttavia di un espediente necessario in quanto suddivide i parametri di privacy differenziale in modo appropriato per essere utilizzati nelle funzioni di aggregazione DP; questa soluzione evita di dover calcolare le singole quote di budget DP ogni volta che viene aggiunta un'operazione DP alla pipeline.

Sono state implementate altre due funzioni per inizializzare il budget che vengono utilizzate in combinazione con le operazioni di tipo specializzato e quindi che non dipendono dalla definizione YAML, queste sfruttano comunque una strategia molto simile: l'utente compone una mappa che associa al nome di ogni operazione un valore numerico che ne indica l'importanza, il significato di questo valore verrà discusso nella sezione relativa al formato di definizione YAML.

3.2 DPYaml - definizione di pipeline DP

Per permettere agli utenti di GoDP di definire una pipeline DP è stato definito DPYaml, un formato di specifica in linguaggio YAML che permette a utenti non esperti di creare distribuzioni di dati differenzialmente private tramite la definizione delle operazioni di aggregazione da applicare a un dataset. Di seguito se ne illustrano le caratteristiche e le parti che lo compongono; in appendice si riporta un esempio di un file DPYaml come riferimento (1).

DPYaml si articola in quattro sezioni:

- configurazione
- budget di privacy
- tipi di dato
- operazioni

Tra questi, le sezioni di configurazione, budget di privacy e operazioni sono indispensabili al funzionamento dell'applicazione. La definizione dei tipi di dato si rende necessaria soltanto nei casi in cui si definiscano delle operazioni che richiedano gestione particolare del tipo di dato da elaborare, come ad esempio un'operazione che calcola la media di un valore aggregato su un campo chiave, al contrario di operazioni come il conteggio di elementi che appartengono a una categoria.

Configurazione

La sezione *configurazione*, in DPYaml denotata dalla chiave `configuration`, è dedicata alla definizione di valori di impostazione della pipeline; in particolare sono richiesti i seguenti parametri:

- `data_dir`: il nome della cartella che contiene il dataset da elaborare;
- `input`: il nome del dataset da elaborare, deve trovarsi all'interno di `data_dir`;
- `output_base_name`: nome base utilizzato per comporre il nome dei file di output generati e
- `id_field`: nome di un campo del dataset.

Il valore associato a `output_base_name` viene utilizzato come template per nominare i file che conterranno i risultati delle elaborazioni, concatenando il nome dell'operazione che ha generato il risultato contenuto.

Il valore dato al campo `id_field` definisce quale attributo del dataset verrà utilizzato dall'applicazione per definire l'unità di privacy differenziale, ovvero quel valore dei record del dataset che identifica univocamente un soggetto; questo valore può comparire più volte nel dataset, indicando la presenza di più record relativi a un determinato soggetto.

Budget di privacy

La sezione del *budget di privacy*, denotata dalla chiave `privacy_budget`, permette di specificare i parametri utilizzati da GoDP per configurare gli algoritmi di privacy differenziale.

I parametri in questione sono `delta` ed `epsilon`; questi valori, insieme al tipo di distribuzione di probabilità usato, specificato tramite il campo `noise_kind`, vengono utilizzati da GoDP per configurare il meccanismo volto a garantire la DP durante l'esecuzione delle operazioni di aggregazione, determinando la quantità di rumore da aggiungere in funzione della sensitività della funzione di aggregazione e del livello di protezione desiderato (2.3).

L'ultimo parametro di questa sezione è denotato dal nome `aggregation_share` e permette di specificare come suddividere il budget di privacy tra le operazioni di aggregazione e selezione delle partizioni ($1 - $`aggregation_share`). In questo modo, è possibile indicare all'applicazione quanta parte del budget di privacy debba essere utilizzata per rendere differenzialmente privata la fase di selezione delle categorie da includere nel risultato (selezione delle partizioni) per evitare eventi distintivi e quanta, invece, debba essere allocata per applicare l'effettiva addizione di rumore al risultato finale (aggregazione).

Operazioni

La sezione `operations` costituisce il nucleo operativo di DPYaml, in quanto permette di definire in modo dichiarativo la sequenza di trasformazioni DP che saranno applicate al dataset di input. Ogni elemento di questa sezione descrive un'operazione di aggregazione DP che andrà a consumare parte del budget di privacy specificato nella sezione `privacy_budget`, specificandone il tipo, i parametri di privacy e i campi del dataset coinvolti. Questo insieme di operazioni è utilizzato da GoDP per configurare la pipeline DP, nella quale ciascuna operazione contribuisce in modo esplicito alla perdita complessiva di privacy.

Ogni operazione è identificata da un nome univoco, specificato tramite il sotto-parametro `name`, che viene utilizzato sia per nominare i file di output contenenti i risultati delle operazioni, sia per scopi di tracciamento interno (e.g. porzione del budget, debug).

Il campo `type` specifica la tipologia di aggregazione da eseguire, scelta tra `count`, `sum` e `mean`, determinando quale funzione del modulo `aggregations` venga invocata; il campo `column` indica il nome dell'attributo del dataset al quale applicare il tipo di operazione specificato.

Un aspetto centrale della sezione `operations` è la possibilità di definire parametri di privacy specifici per ogni tipo di aggregazione delle operazioni aggiunte

alla pipeline. Compilando la sezione `privacy_params` con appositi valori specifici per ogni tipo di operazione, è possibile calibrare la quantità di rumore aggiunto ai risultati delle operazioni. I parametri in questione descrivono proprietà intrinseche rilevanti del dataset utilizzato. Segue una descrizione dei parametri e di come essi influenzino la quantità di rumore aggiunto ai risultati e, di conseguenza, il livello di compromesso tra perdita di privacy e utilità dei dati.
L'operazione `count` supporta i seguenti parametri:

- `max_contributions` indica il numero massimo di volte che un identificatore di privacy può contribuire a un singolo conteggio (o il numero massimo che esso può aggiungere a un singolo conteggio); se un identificatore contribuisce al conteggio un numero di volte maggiore di `max_contributions`, l'eccesso viene ignorato;
- `max_categories_contributed` indica il numero massimo di partizioni distinte al quale un singolo identificatore di privacy può contribuire, se un identificatore è associato a un numero maggiore di categorie un insieme casuale di valori viene ignorato.

Le operazioni `mean_per_key` e `sum_per_key` richiedono il parametro aggiuntivo `key_column` che permette di indicare quale attributo del dataset utilizzare come chiave sulla quale effettuare la media o la somma, ad esempio una operazione di media per chiave con `column = "Age"` e `key_column = "MedicalCondition"` risulta nella media di età rilevata per ogni diagnosi di una certa condizione medica. I parametri di privacy per questo tipo di operazioni sono i seguenti:

- `max_categories_contributed` ha lo stesso significato descritto per l'operazione di `count`;
- `max_contributions_per_category` indica il numero massimo di contribuzioni da un dato identificatore di privacy a una singola categoria (o chiave) sulla quale viene effettuate la media;
- `min_value, max_value` definiscono l'intervallo entro il quale un valore utilizzato nella media deve essere compreso, un valore dell'attributo che contribuisce alla media che è inferiore a `min_value` viene vincolato al minimo dell'intervallo, viceversa per il limite superiore.

La scelta di questi parametri comporta un compromesso fondamentale: l'aumento dei valori di questi parametri di privacy (o all'aumentare della dimensione dell'intervallo nel caso di `min_value` e `max_value`), corrisponde una diminuzione della perdita di informazione, dato che una minore porzione dei dati verrebbe ignorata o distorta, ma anche una quantità maggiore di rumore aggiunta ai risultati.

I parametri di privacy influenzano la quantità di rumore aggiunto in quanto modificano i valori effettivi degli attributi utilizzati dall'operazione DP, modificando quindi la sensitività della funzione applicata al dataset con conseguente aumento o

diminuzione del rumore aggiunto (2.12).

4 Validazione sperimentale: Attacco di de-anonimizzazione

In questo capitolo si procede ad illustrare un caso di utilizzo di GoDP con l'obiettivo di dimostrare l'efficacia della privacy differenziale contro attacchi di de-anonimizzazione. L'esperimento simula uno scenario realistico in cui un attaccante tenta di dedurre informazioni sensibili su un individuo specifico a partire da aggregazioni su un dataset, confrontando i risultati ottenuti con e senza l'applicazione di meccanismi differenzialmente privati.

4.1 Threat model

L'esperimento si basa sul modello di privacy differenziale *centralizzato*, nel quale un *curatore* fidato ha accesso al database in chiaro e agisce come intermediario tra la sorgente dei dati e gli analisti che desiderano effettuare query aggregate sul dataset. Il curatore riceve le richieste di aggregazione, applica un meccanismo DOP opportunamente calibrato e restituisce il risultato protetto dalle garanzie della privacy differenziale.

Nel contesto di questo modello di applicazione della DP, si identificano quattro attori principali:

- **dataset originale**, contenente i dati sensibili dei dipendenti di un'azienda in chiaro e reso accessibile esclusivamente al curatore fidato;
- **curatore**;
- **analista** e
- **attaccante**.

L'*analista* è un soggetto che utilizza il sistema di interrogazione del database allo scopo di analizzare i dati contenuti, mentre l'*attaccante* è un analista malintenzionato che ha a disposizione informazioni ausiliarie sulla vittima raccolte da fonti esterne, come ad esempio piattaforme di social networking, o ottenute tramite tecniche di social engineering. Il *curatore* è l'entità fidata che implementa e applica

il meccanismo DP: riceve le query, aggiunge rumore calibrato e restituisce risultati differenzialmente privati.

Il threat model si basa su alcune assunzioni fondamentali:

- il curatore è fidato e non collude con l'attaccante
- l'attaccante non può manomettere o corrompere il sistema che implementa il meccanismo DP
- gli analisti possono osservare esclusivamente i risultati differenzialmente privati; non hanno accesso diretto ai dati grezzi del dataset
- l'attaccante può avere accesso a informazioni ausiliarie sulla vittima ottenute da fonti esterne
- l'attaccante conosce la struttura del dataset, ovvero gli attributi che il dataset contiene; queste informazioni sono necessariamente pubbliche per permettere un accesso efficace da parte degli analisti

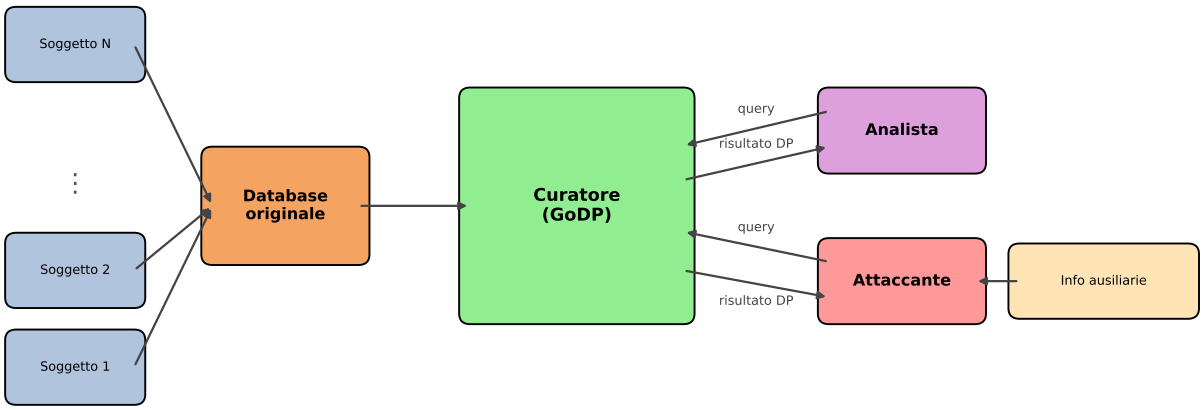


Figura 4.1: Modello DP centralizzata

4.2 Difference attack

Il *difference attack* è una tecnica di de-anonimizzazione che permette a un attaccante di dedurre informazioni sensibili su un singolo individuo combinando i risultati di più query aggregate. L'attacco sfrutta il fatto che le aggregazioni esatte permettono di isolare il contributo di un individuo specifico attraverso sottrazioni successive. [3]

Sia D un dataset a f una funzione di aggregazione (e.g. somma, conteggio, media...); l'attaccante identifica due sottoinsiemi:

- D_{include}, che include la vittima target e
- D_{exclude}, lo stesso sottoinsieme senza la vittima

Il valore associato alla vittima può essere calcolato come:

$$\text{record}_{\text{vittima}} = f(D_{\text{include}} - D_{\text{exclude}}) \tag{4.1}$$

A scopo di illustrare l'efficacia del difference attack, si considera un sistema che sfrutta una tecnica di protezione basata su una *soglia minima di aggregazione*, questo metodo impone che il sistema rifiuti di restituire qualsiasi risultato quando il numero di record che soddisfa i criteri della query è inferiore a una costante k. Questo meccanismo, noto come *suppression threshold*, è ampiamente utilizzato in sistemi statistici reali come protezione contro la re-identificazione [**kThreshold**].

In presenza di tale meccanismo, una query che filtra per tutti gli attributi noti della vittima verrebbe rifiutata dal sistema, in quanto restituirebbe un singolo record ($1 < k$). L'attaccante è quindi costretto a ricorrere al difference attack: interrogando il sistema con query che coinvolgono gruppi sufficientemente numerosi da superare la soglia k, e calcolando differenze tra i risultati, può aggirare la protezione e isolare il contributo della vittima. Questo scenario dimostra come la soglia minima di aggregazione, pur offrendo un certo livello di protezione, sia vulnerabile ad attacchi che sfruttano conoscenze ausiliarie sui membri del dataset. Per gli scopi di questo esperimento si utilizzerà una costante $k = 5$ (1% dataset).

Tenendo conto delle informazioni ausiliarie a disposizione dell'attaccante, egli procede a interrogare il sistema tramite una serie di query specifiche che gli permettono di restringere progressivamente l'insieme dei record considerati fino a isolare il record contenente le informazioni della vittima.

L'attaccante inizia interrogando il sistema per ottenere la somma totale degli stipendi del reparto della vittima.

$$S_1 = \text{SUM(Salary) WHERE Department} = \text{'Engineering'} \tag{4.2}$$

Successivamente, esclude i dipendenti che hanno la stessa età della vittima con una seconda query.

$$S_2 = \text{SUM(Salary) WHERE Department} = \text{'Engineering'} \ \wedge \ \text{Age} \neq 35 \tag{4.3}$$

La differenza tra i due dataset isola i dipendenti del reparto Engineering che hanno 35 anni: $S_{\text{age=35}} = S_1 - S_2$.

A questo punto, l'attaccante effettua una query che esclude il genere della vittima dal gruppo isolato:

$$S_3 = \text{SUM(Salary) WHERE Department} = \text{'Engineering'} \wedge \text{Age} = 35 \wedge \text{Gender} \neq \text{'F'} \tag{4.4}$$

Se la combinazione di attributi (Department=Engineering, Age=35, Gender=F) identifica univocamente la vittima nel dataset, lo stipendio può essere dedotto esattamente:

$$\text{Stipendio}_{\text{vittima}} = S_{\text{age=35}} - S_3 \tag{4.5}$$

Questo attacco può avere successo se vengono verificate le seguenti condizioni:

- **unicità della vittima**: la combinazione di attributi noti all'attaccante deve identificare univocamente una sola persona nel dataset
- **query esatte**: le aggregazioni devono restituire valori esatti senza aggiunta di rumore, permettendo calcoli precisi

Senza le garanzie di protezione offerte dalla DP, l'attaccante è in grado di risalire allo stipendio esatto della sua vittima.

La Figura 4.2 illustra il procedimento dell'attacco, mostrando come le sottrazioni successive permettano di isolare progressivamente lo stipendio della singola vittima a partire da aggregazioni su gruppi di numerosità elevata.

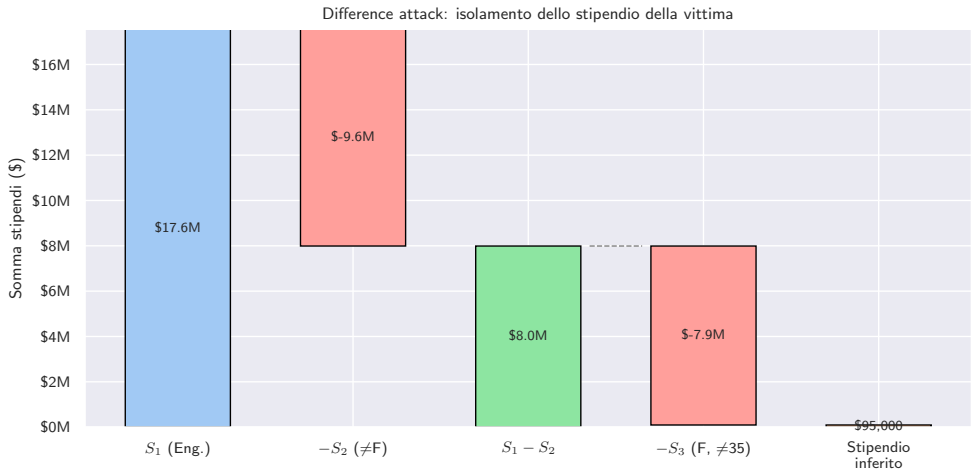


Figura 4.2: Fasi del difference attack: le sottrazioni successive isolano il contributo della vittima.

4.2.1 Dataset sperimentale

Per condurre l'esperimento, è stato generato un dataset sintetico che simula i dati dei dipendenti di un'azienda. L'obiettivo principale è dimostrare l'efficacia della privacy differenziale contro il differencing attack in condizioni controllate e verificabili. Questo richiede la garanzia che siano soddisfatte le condizioni necessarie al successo dell'attacco: la vittima deve essere univocamente identificabile tramite la combinazione di attributi noti all'attaccante; questa caratteristica, unita alla possibilità di calibrare con precisione i parametri di privacy, rende un dataset sintetico adatto agli scopi di questo esperimento.

Il dataset utilizzato è composto dalle informazioni di circa 500 dipendenti dove ogni record rappresenta un singolo dipendente, caratterizzato dai seguenti attributi:

- `ID`
- `Department`
- `Age`
- `Gender`
- `YearsExperience`
- `Salary`, stipendio annuale

Per rendere la generazione del dataset realistica, sono state definite apposite distribuzioni di probabilità per l'assegnazione dei reparti e soglie plausibili per i valori di età, esperienza e stipendio, calibrandoli in base al reparto di appartenenza. Per le configurazioni esatte, consultare il codice allegato.

Si riportano di seguito le informazioni relative alla vittima che l'attaccante ha ottenuto tramite l'analisi di fonti di dati ausiliarie:

- genere: femminile
- reparto lavorativo: engineering
- età: 35 anni

Queste sono informazioni facilmente ottenibili tramite piattaforme come Linkedin o altri social network; queste informazioni, combinate con la conoscenza della struttura del dataset, permettono all'attaccante di definire una serie di query da eseguire sul database per condurre un *difference attack*.

In aggiunta a queste informazioni, si è deciso di impostare lo stipendio della vittima a $95,000: questo è il valore che l'attaccante mira a de-anonimizzare.

4.3 Configurazione dell'esperimento e risultati

L'esperimento valuta l'efficacia della protezione DP al variare del parametro ε, mantenendo costante $\delta = 1e-5$ ($<< 1/n$); i valori di epsilon scelti per eseguire l'esperimento corrispondono a quattro livelli di privacy: alto ($\varepsilon = 0.5$), moderato ($\varepsilon = 5$), medio ($\varepsilon = 10$) e basso ($\varepsilon = 15$).

La scelta del valore di ε rappresenta un problema aperto nella comunità della privacy differenziale [**epsilonchoice**]. Non esistono linee guida universali e la scelta dipende fortemente dalla sensibilità dei dati, così come dal dominio: a seconda del tipo di dati trattati, quantità diverse di rumore possono essere più o meno accettabili. Questo concetto verrà discusso ulteriormente in seguito.

L'aggregazione differenzialmente privata è configurata tramite il formato DPYaml, specificando il meccanismo di rumore Laplace (`noise_kind: laplace`), l'identificatore di privacy (`id_field: ID`) e la suddivisione del budget (`aggregation_-`

`share: 0.9`), che destina il 90% del budget all'aggregazione e il restante 10% alla selezione della partizione.

Per eseguire il difference attack attraverso GoDP, ciascuna delle tre query dell'attacco viene configurata con una specifica DPYaml con i filtri corrispondenti. Tramite la sezione `filters` di DPYaml si specificano condizioni di selezione applicate al dataset prima di effettuare l'aggregazione DP specificata, permettendo all'attaccante di eseguire le stesse query che effettuerebbe sul database non protetto.

Per ogni valore di ε, sono state generate tre configurazioni distinte, una per ciascuna query S_1, S_2, S_3, per un totale di 12 esecuzioni (4 valori di $\varepsilon \times 3$ query).

L'operazione di aggregazione utilizzata (`sum_per_key`) calcola la somma degli stipendi per reparto. I parametri di configurazione specificano che ogni dipendente appartiene a un solo reparto e contribuisce (`max_categories_contributed: 1`) e contribuisce una sola volta (`max_contributions_per_category: 1`), con valori di stipendio compresi tra \$30,000 e \$200,000.

La sensitività globale dell'operazione di somma è determinata dall'intervallo dei possibili valori:

$$\Delta f = \texttt{max_value} - \texttt{min_value} = 200{,}000 - 30{,}000 = 170{,}000 \tag{4.6}$$

Il parametro di scala della distribuzione di Laplace, considerando che il 90% del budget è allocato all'aggregazione, è:

$$b = \frac{\Delta f}{\varepsilon_{\text{agg}}} = \frac{170{,}000}{0.9\varepsilon} = \frac{188{,}889}{\varepsilon} \tag{4.7}$$

In assenza di protezione DP, l'attacco di differenza ha successo con precisione del 100%, l'attaccante riesce a dedurre esattamente lo stipendio della vittima, identificando un unico individuo che corrisponde alla combinazione di attributi nota. Questo risultato dimostra la vulnerabilità della combinazione delle aggregazioni non protette: utilizzando esclusivamente query aggregate, l'attaccante è in grado di determinare con certezza il valore sensibile di un individuo specifico.

4.3.1 Attacco senza protezione DP

In assenza di protezione differenzialmente privata, il sistema restituisce i risultati esatti delle aggregazioni. Le tre query dell'attaccante producono i seguenti risultati:

- S_1 = \$17,558,674, somma degli stipendi di 157 dipendenti del reparto Engineering
- S_2 = \$9,568,613, somma degli stipendi di 87 dipendenti maschi del reparto Engineering

- S_3 = \$7,895,061, somma degli stipendi di 69 donne nel reparto Engineering con età diversa da 35

Applicando il calcolo dell'equazione (4.5):

$$\text{Stipendio}_{\text{vittima}} = (S_1 - S_2) - S_3 = (17{,}558{,}674 - 9{,}568{,}613) - 7{,}895{,}061 = \$95{,}000$$

L'attaccante riesce a dedurre lo stipendio esatto della vittima, coincidente con il valore reale di \$95,000. L'attacco ha successo con precisione del 100%, dimostrando che le aggregazioni esatte non offrono alcuna protezione.

4.3.2 Attacco con protezione DP

Si ripete lo stesso attacco attraverso GoDP, che applica il meccanismo di Laplace alle aggregazioni. I risultati sono riassunti nella Tabella 4.1.

Tabella 4.1: Risultati del difference attack con protezione DP al variare di ε.

	$\varepsilon = 0.5$	$\varepsilon = 5$	$\varepsilon = 10$	$\varepsilon = 15$
S_1 DP	\$17,302,307	\$17,549,307	\$17,571,821	\$17,571,068
S_2 DP	rimossa	\$9,625,483	\$9,581,561	\$9,556,128
S_3 DP	rimossa	\$7,879,474	\$7,899,952	\$7,899,090
Stipendio inferito	–	\$44,350	\$90,308	\$115,850
Errore assoluto	–	\$50,650	\$4,692	\$20,850
Errore relativo	–	53.3%	4.9%	21.9%
Protezione	massima	moderata	sufficiente	sufficiente

$\varepsilon = 0.5$: rimozione della partizione

Per $\varepsilon = 0.5$, il meccanismo di partition selection rimuove completamente le partizioni delle query S_2 e S_3: i risultati di queste query non vengono restituiti dal sistema. In questo caso il budget ε destinato alla partition selection ($0.1 \times 0.5 = 0.05$) è insufficiente a mantenere le partizioni che contengono un numero ridotto di record, impedendo del tutto il calcolo delle differenze necessarie all'attacco.

Questo risultato evidenzia come la partition selection agisca come un ulteriore livello di protezione: per valori molto bassi di ε, il meccanismo non si limita ad aggiungere rumore alle aggregazioni, ma impedisce la pubblicazione stessa dei risultati di query che coinvolgono gruppi di dimensione ridotta. L'attaccante non è in grado di eseguire il difference attack in quanto non dispone dei dati necessari al calcolo. Questo fenomeno è consistente con le garanzie teoriche della DP e mostra come un valore di ε molto stringente possa portare a una perdita completa di utilità dei dati a fronte di una protezione massima.

$\varepsilon = 5$: protezione moderata

Per $\varepsilon = 5$ tutte le query restituiscono risultati, ma il rumore composto attraverso le sottrazioni successive porta a un'inferenza di \$44,350, con un errore del 53.3% rispetto allo stipendio reale di \$95,000. La composizione del rumore è un effetto fondamentale: poiché lo stipendio è calcolato come $(S_1 - S_2) - S_3$, il rumore delle tre query si somma. In questo caso, i singoli rumori sulle query ammontano a circa \$9,367 su S_1, \$56,870 su S_2 e \$15,587 su S_3; il rumore composto sul risultato finale è la somma algebrica di questi contributi che porta a un errore significativo. Il livello di protezione è moderato: l'attaccante ottiene un valore che è lontano da quello reale.

$\varepsilon = 10$ e $\varepsilon = 15$: protezione ridotta

Per $\varepsilon = 10$ l'attaccante inferisce un valore di \$90,308, con un errore del 4.9%; per $\varepsilon = 15$ il valore inferito è \$115,850, con un errore del 21.9%. In entrambi i casi il rumore aggiunto alle singole query è ridotto (ordine di grandezza \$4,000–\$13,000), ma l'errore finale dipende dalla composizione algebrica dei rumori che può portare sia a sovrastime che a sottostime del valore reale.

Questi valori di ε offrono una protezione limitata: lo stipendio inferito è sufficientemente vicino al valore reale da permettere all'attaccante di ottenere un'approssimazione ragionevole del dato sensibile.

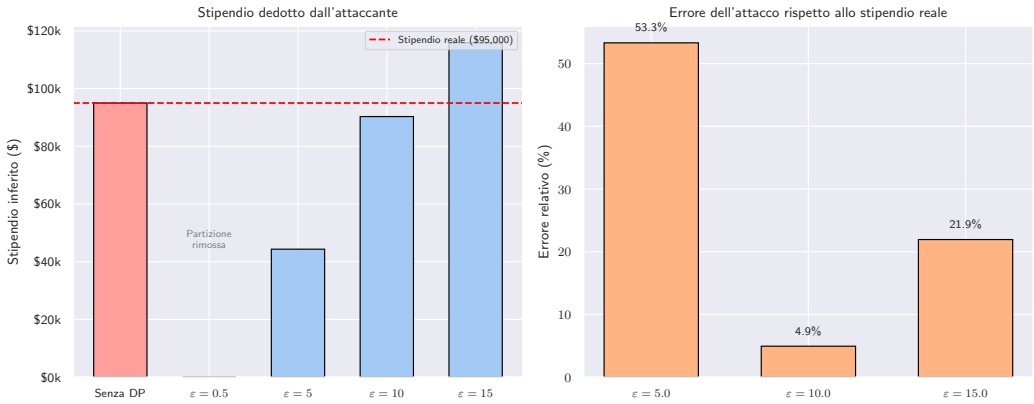


Figura 4.3: Confronto tra i risultati dell'attacco con e senza protezione DP (sinistra) e errore relativo al variare di ε (destra).

4.3.3 Analisi del rumore

Il rumore aggiunto dal meccanismo di Laplace a ciascuna query è determinato dal parametro di scala $b = \Delta f / \varepsilon_{\text{agg}}$ (equazione (4.7)). La Tabella 4.2 riporta il rumore assoluto misurato su ciascuna query per i valori di ε dove tutte le query sono disponibili.

Tabella 4.2: Rumore assoluto ($) aggiunto alle singole query dal meccanismo di Laplace.

	b (teorico)	|Rumore S_1|	|Rumore S_2|	|Rumore S_3|
$\varepsilon = 5$	\$37,778	\$9,367	\$56,870	\$15,587
$\varepsilon = 10$	\$18,889	\$13,147	\$12,948	\$4,891
$\varepsilon = 15$	\$12,593	\$12,394	\$12,485	\$4,029

Si osserva che al diminuire di ε il parametro di scala b cresce, portando a realizzazioni di rumore di ampiezza mediamente maggiore; per $\varepsilon = 5$ il valore atteso del rumore assoluto è $\mathbb{E}[|L|] = b = \$37{,}778$, rendendo improbabile un'inferenza accurata.

La Figura 4.4 mostra il rumore registrato per ciascuna query al variare di ε, insieme all'andamento teorico del parametro di scala. Si noti come il rumore osservato sia consistente con il parametro di scala atteso per la distribuzione di Laplace.

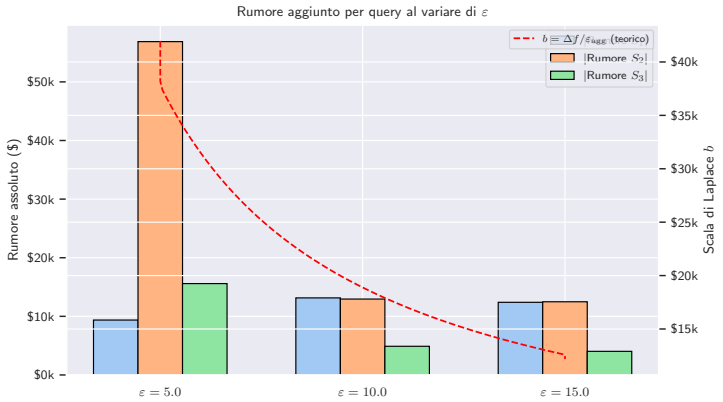


Figura 4.4: Rumore assoluto per query al variare di ε e confronto con il parametro di scala teorico $b = \Delta f/\varepsilon_{\text{agg}}$.

Un aspetto critico dell'attacco è la *composizione del rumore* attraverso le operazioni di sottrazione. Lo stipendio inferito è calcolato come $(S_1 - S_2) - S_3$; denotando con η_i il rumore sulla query S_i, il rumore composto sul risultato finale è:

$$\eta_{\text{tot}} = \eta_1 - \eta_2 - \eta_3 \tag{4.8}$$

Poiché le variabili η_i sono indipendenti e distribuite secondo $\text{Lap}(0, b)$, la varianza del rumore composto è:

$$\text{Var}(\eta_{\text{tot}}) = 3 \cdot \text{Var}(\eta_i) = 3 \cdot 2b^2 = 6b^2 \tag{4.9}$$

Questo significa che la deviazione standard del rumore composto è $\sigma_{\text{tot}} = b\sqrt{6} \approx 2.45b$, indicando come l'esecuzione di tre query amplifichi significativamente l'incertezza rispetto a una singola query.

4.4 Discussione

I risultati sperimentali illustrano il trade-off fondamentale tra privacy e utilità che caratterizza la privacy differenziale.

4.4.1 Efficacia della protezione

L'esperimento mostra un comportamento coerente con le proprietà teoriche della DP: al diminuire di ε, la protezione contro l'attacco di de-anonimizzazione aumenta. Per $\varepsilon = 0.5$ l'attacco è completamente neutralizzato dalla rimozione delle partizioni; per $\varepsilon = 5$ il rumore composto rende l'inferenza inaffidabile con un errore del 53.3%. Per valori più elevati di ε (10 e 15), la protezione si riduce e l'attaccante riesce a ottenere approssimazioni più vicine al valore reale.

Questo comportamento è consistente con la definizione di ε-DP: al crescere di ε il rapporto tra le probabilità di osservare un dato output con o senza la vittima nel dataset può divergere maggiormente, permettendo all'attaccante di fare inferenze più precise.

4.4.2 Trade-off privacy-utilità

La scelta del valore di ε deve bilanciare la protezione degli individui con l'utilità dei dati aggregati. Per $\varepsilon = 0.5$, sebbene la protezione sia massima, il sistema non restituisce risultati per gran parte delle query, rendendo i dati inutilizzabili per scopi analitici legittimi. Per $\varepsilon = 15$, i dati hanno un'utilità elevata ma la protezione può risultare insufficiente per dati altamente sensibili come gli stipendi.

Il valore di $\varepsilon = 5$ rappresenta in questo scenario un compromesso ragionevole: tutte le query restituiscono risultati (garantendo un certo livello di utilità), ma il rumore composto è sufficiente a impedire un'inferenza accurata del dato sensibile.

4.4.3 Ruolo della selezione di partizione

I risultati evidenziano il ruolo della partition selection come meccanismo di protezione complementare al rumore. Il budget allocato alla partition selection (10% del totale) oltre a non diminuire la protezione dell'aggregazione agisce come un filtro che esclude dalla pubblicazione le aggregazioni su gruppi di dimensione ridotta. Per $\varepsilon = 0.5$, questo meccanismo è il fattore determinante nella protezione della vittima: le query S_2 e S_3, che filtrano per attributi specifici e producono gruppi più piccoli, vengono rimosse dal meccanismo prima ancora che il rumore venga applicato.

4.4.4 Limiti dell'esperimento

L'esperimento presenta alcuni limiti che è opportuno evidenziare. In primo luogo, i risultati riportati corrispondono a una singola esecuzione del meccanismo DP: poiché il rumore è stocastico, esecuzioni diverse producono risultati diversi e l'errore osservato può variare significativamente. Un'analisi statistica su molteplici esecuzioni permetterebbe di caratterizzare la distribuzione degli errori in modo più accurato.

In secondo luogo, il dataset è sintetico e progettato per garantire le condizioni di successo dell'attacco (unicità della vittima). In contesti reali, fattori come la dimensione del dataset, la distribuzione degli attributi e la complessità dello schema possono influenzare sia la fattibilità dell'attacco sia l'efficacia della protezione DP.

Infine, l'attaccante dispone di conoscenza perfetta della struttura del dataset, un'assunzione che rappresenta uno scenario pessimistico rispetto a molti contesti reali. Questa assunzione è tuttavia metodologicamente corretta: le garanzie della DP sono progettate per resistere anche ad attaccanti con conoscenza massimale del database, come mostrato nella formulazione teorica del Capitolo 2.

Appendices

A | Derivazione equazioni

Passaggi 2.6:

$$e^{-\varepsilon} \cdot \frac{\Pr[D = D_{in}]}{\Pr[D = D_{out}]} \leq \frac{\Pr[D = D_{in} | \mathcal{M}(D) = O]}{\Pr[D = D_{out} | \mathcal{M}(D) = O]} \leq e^{\varepsilon} \cdot \frac{\Pr[D = D_{in}]}{\Pr[D = D_{out}]}$$

$$e^{-\varepsilon} \cdot \frac{\Pr[D = D_{in}]}{1 - \Pr[D = D_{in}]} \leq \frac{\Pr[D = D_{in} | \mathcal{M}(D) = O]}{1 - \Pr[D = D_{in} | \mathcal{M}(D) = O]} \leq e^{\varepsilon} \cdot \frac{\Pr[D = D_{in}]}{1 - \Pr[D = D_{in}]}$$

Si considera solo il lato destro dato che le due espressioni sono simmetriche.

$$\Pr[D = D_{in} | \mathcal{M}(D) = O] \leq \frac{e^{\varepsilon} \cdot \Pr[D = D_{in}]}{1 - \Pr[D = D_{in}]} - \Pr[D = D_{in} | \mathcal{M}(D) = O] \cdot \frac{e^{\varepsilon} \cdot \Pr[D = D_{in}]}{1 - \Pr[D = D_{in}]}$$

$$\Pr[D = D_{in} | \mathcal{M}(D) = O] \cdot \left(\frac{1 - \Pr[D = D_{in}] + e^{\varepsilon} \cdot \Pr[D = D_{in}]}{1 - \Pr[D = D_{in}]} \right) \leq \frac{e^{\varepsilon} \cdot \Pr[D = D_{in}]}{1 - \Pr[D = D_{in}]}$$

$$\Pr[D = D_{in} | \mathcal{M}(D) = O] \leq \frac{e^{\varepsilon} \cdot \Pr[D = D_{in}]}{1 - \Pr[D = D_{in}] + e^{\varepsilon} \cdot \Pr[D = D_{in}]}$$

Sostituendo $e^{-\varepsilon}$ a ogni istanza di e^{ε} si ottiene il lato sinistro, mettendo tutto insieme:

$$\frac{\Pr[D = D_{in}]}{e^{\varepsilon} + (1 - e^{\varepsilon}) \cdot \Pr[D = D_{in}]} \leq \Pr[D = D_{in} | \mathcal{M}(D) = O] \leq \frac{e^{\varepsilon} \cdot \Pr[D = D_{in}]}{1 - (e^{\varepsilon} - 1) \cdot \Pr[D = D_{in}]}$$

B | Dimostrazioni

Dimostrazione 2.9: Si definisca $x \in \mathbb{N}^k$ e $y \in \mathbb{N}^k$ tali che $||x - y||_1 \leq 1$, e si consideri una funzione $f\colon \mathbb{N}^{|\mathcal{X}|} \to \mathbb{R}^k$. Si indichi con p_x la funzione di densità di probabilità $\mathcal{M}_L(x, f, \varepsilon)$ e con p_y la funzione di densità di probabilità $\mathcal{M}_L(y, f, \varepsilon)$. Comparando le due funzioni a un punto arbitrario $z \in \mathbb{R}^k$:

$$\begin{aligned}
\frac{p_x(z)}{p_y(z)} &= \prod_{i=1}^{k} \left(\frac{\exp\left(-\frac{\varepsilon|f(x)_i - z_i|}{\Delta f}\right)}{\exp\left(-\frac{\varepsilon|f(y)_i - z_i|}{\Delta f}\right)} \right) \\
&= \prod_{i=1}^{k} \exp\left(\frac{\varepsilon(|f(y)_i - z_i| - |f(x)_i - z_i|)}{\Delta f} \right)
\end{aligned}$$

Applicando la disuguaglianza triangolare:

$$\begin{aligned}
&\leq \prod_{i=1}^{k} \exp\left(\frac{\varepsilon|f(x)_i - f(y)_i|}{\Delta f} \right) \\
&= \exp\left(\frac{\varepsilon \cdot ||f(x) - f(y)||_1}{\Delta f} \right)
\end{aligned}$$

Per la definizione di sensitività:

$$\leq e^{\varepsilon}$$

Dimostrazione 2.12: Si consideri un meccanismo A e un parametro $\varepsilon \geq 0$, il minor parametro δ tale che il meccanismo A è (ε, δ)-DP è definito come:

$$\delta_S = \Pr[A(D_{in}) \in S] - e^{\varepsilon} \cdot \Pr[A(D_{out} \in S]$$

La definizione è soddisfatta se $\delta_S \leq \delta$, quindi si definisce δ:

$$\delta = \max_S (\Pr[A(D_{in}) \in S] - e^{\varepsilon} \cdot \Pr[A(D_{out} \in S])$$

Sono di interesse soltanto output O tali che $\Pr[A(D_{in}) = O] > e^{\varepsilon} \cdot \Pr[A(D_{out}) = O]$, in quanto tutti gli altri output renderebbero δ_{max} più piccolo. L'insieme S che

minimizza questa quantità è:

$$S_{max} := \{O | \Pr[A(D_{in}) = O] > e^{\varepsilon} \cdot \Pr[A(D_{out}) = O]\}$$

Riscrivendo la definizione di δ precedente sostituendo con il nuovo insieme S_{max} si ottiene:

$$\begin{aligned}
\delta &= \Pr[A(D_{in}) \in S_{max}] - e^{\varepsilon} \cdot \Pr[A(D_{out}) \in S_{max}] \\
&= \sum_{O \in S_{max}} (\Pr[A(D_{in}) = O] - e^{\varepsilon} \cdot \Pr[A(D_{out}) = O]) \\
&= \sum_{O \in S_{max}} \Pr[A(D_{in}) = O] \left(1 - \frac{e^{\varepsilon}}{e^{\mathcal{L}_{D_{in},D_{out}}(O)}}\right)
\end{aligned}$$

Invece di sommare soltanto gli output $O \in S_{max}$, possiamo riscrivere considerando tutti gli output $O \in S$ azzerando quelli che non appartengono a S_{max}.

$$\delta = \sum_{O} \Pr[A(D_{in}) = O] \max\left(0, 1 - \frac{e^{\varepsilon}}{e^{\mathcal{L}_{D_{in},D_{out}}(O)}}\right)$$

L'espressione sopra è un valore atteso pesato sulla probabilità che $A(D_{in}) = O$:

$$\delta = \mathbb{E}_{O \sim A(D_{in})} \left[\max\left(0, 1 - \frac{e^{\varepsilon}}{e^{\mathcal{L}_{D_{in},D_{out}}(O)}}\right)\right]$$

che corrisponde all'area tra la curva $e^{\varepsilon}/e^{\mathcal{L}}$ e la retta $y = 1$.

C | Codice sorgente

```yaml
pipelinedp:
  configuration:
    data_dir: "data"
    input: "healthcare_cleaned.csv"
    output_base_name: "output.csv"
    id_field: "Name"

  privacy_budget:
    noise_kind: "laplace"
    delta: 1e-5
    epsilon: 1
    aggregation_share: 0.5
  types:
    - column: "Age"
      type: int
    - column: "DateOfAdmission"
      type: date
    - column: "BillingAmount"
      type: float

  operations:
    - name: CountTestResults
      type: count
      column: "TestResults"
      importance: 1
      privacy_params:
        max_contributions: 5
        max_categories_contributed: 25
    - name: CountConditions
      type: count
      column: "MedicalCondition"
      importance: 2
      privacy_params:
        max_contributions: 15
        max_categories_contributed: 6
    - name: MeanAgeByCondition
```

```yaml
      type: mean_per_key
      column: "Age"
      key_column: "MedicalCondition"
      importance: 3
      privacy_params:
        max_categories_contributed: 6
        max_contributions_per_category: 8
        min_value: 5
        max_value: 95
    - name: SumExpenseByCondition
      type: sum_per_key
      column: "BillingAmount"
      key_column: "MedicalCondition"
      importance: 3
      privacy_params:
        max_categories_contributed: 6
        min_value: 0
        max_value: 60000
    - name: MeanBillByCondition
      type: mean_per_key
      column: "BillingAmount"
      key_column: "MedicalCondition"
      importance: 3
      privacy_params:
        max_categories_contributed: 6
        max_contributions_per_category: 8
        min_value: 0
        max_value: 100000
```

Bibliografia

[1] Michael Hilton e Cal. «Differential Privacy : A Historical Survey». In: 2012. URL: https://api.semanticscholar.org/CorpusID:16861132.

[2] Cynthia Dwork. «Differential Privacy: A Survey of Results». In: *Theory and Applications of Models of Computation—TAMC*. Vol. 4978. Lecture Notes in Computer Science. Springer Verlag, 2008, pp. 1–19. URL: https://www.microsoft.com/en-us/research/publication/differential-privacy-a-survey-of-results/.

[3] Cynthia Dwork e Aaron Roth. «The Algorithmic Foundations of Differential Privacy». In: *Foundations and Trends® in Theoretical Computer Science* 9.3–4 (2014), pp. 211–407. ISSN: 1551-305X. DOI: 10.1561/0400000042. URL: http://dx.doi.org/10.1561/0400000042.

[4] Experian. «Data Breach Industry Forecast». In: (2015).

[5] Wei-Han Lee et al. *Blind De-anonymization Attacks using Social Networks*. 2018. arXiv: 1801.05534 [cs.SI]. URL: https://arxiv.org/abs/1801.05534.

[6] Arvind Narayanan e Vitaly Shmatikov. «Robust De-anonymization of Large Sparse Datasets». In: *2008 IEEE Symposium on Security and Privacy (sp 2008)*. 2008, pp. 111–125. DOI: 10.1109/SP.2008.33.

[7] Sébastien Gambs, Marc-Olivier Killijian e Miguel Núñez del Prado Cortez. «De-anonymization Attack on Geolocated Data». In: *2013 12th IEEE International Conference on Trust, Security and Privacy in Computing and Communications*. 2013, pp. 789–797. DOI: 10.1109/TrustCom.2013.96.

[8] Cynthia Dwork et al. «Calibrating Noise to Sensitivity in Private Data Analysis». In: *Theory of Cryptography*. A cura di Shai Halevi e Tal Rabin. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006. ISBN: 978-3-540-32732-5.

[9] Tore Dalenius. *Finding a Needle In a Haystack*. URL: https://www.scb.se/contentassets/ca21efb41fee47d293bbee5bf7be7fb3/finding-a-needle-in-a-haystack-or-identifying-anonymous-census-records.pdf.

[10] Ninghui Li, Tiancheng Li e Suresh Venkatasubramanian. «t-Closeness: Privacy Beyond k-Anonymity and l-Diversity». In: *2007 IEEE 23rd International Conference on Data Engineering*. 2007, pp. 106–115. DOI: 10.1109/ICDE.2007.367856.

[11] Aloni Cohen. «Attacks on Deidentification's Defenses». In: *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, ago. 2022, pp. 1469–1486. ISBN: 978-1-939133-31-1. URL: https://www.usenix.org/conference/usenixsecurity22/presentation/cohen.

[12] Peter Kairouz, Sewoong Oh e Pramod Viswanath. *The Composition Theorem for Differential Privacy*. 2015. URL: https://proceedings.mlr.press/v37/kairouz15.pdf.

[13] Apple Differential Privacy Team. *Learning with Privacy at Scale - Apple Machine Learning Research*. [Online; accessed 2025-02-04]. URL: https://machinelearning.apple.com/research/learning-with-privacy-at-scale.

[14] Ahmet Aktay et al. *Google COVID-19 Community Mobility Reports: Anonymization Process Description (version 1.1)*. 2020. arXiv: 2004.04145 [cs.CR]. URL: https://arxiv.org/abs/2004.04145.

[15] Ryan Rogers et al. *LinkedIn's Audience Engagements API: A Privacy Preserving Data Analytics System at Scale*. 2020. arXiv: 2002.05839 [cs.CR]. URL: https://arxiv.org/abs/2002.05839.

[16] Mrinmoy Roy et al. *Machine Learning Applications In Healthcare: The State Of Knowledge and Future Directions*. 2023. arXiv: 2307.14067 [cs.LG]. URL: https://arxiv.org/abs/2307.14067.

[17] Yuqi Nie et al. *A Survey of Large Language Models for Financial Applications: Progress, Prospects and Challenges*. 2024. arXiv: 2406.11903 [q-fin.GN]. URL: https://arxiv.org/abs/2406.11903.

[18] Yang Li, Thilina Ranbaduge e Kee Siong Ng. *Privacy Technologies for Financial Intelligence*. 2024. arXiv: 2408.09935 [cs.CR]. URL: https://arxiv.org/abs/2408.09935.

[19] Alejandro Guerra-Manzanares et al. «Privacy-Preserving Machine Learning for Healthcare: Open Challenges and Future Perspectives». In: *Trustworthy Machine Learning for Healthcare*. Springer Nature Switzerland, 2023, 25–40. ISBN: 9783031395390. DOI: 10.1007/978-3-031-39539-0_3. URL: http://dx.doi.org/10.1007/978-3-031-39539-0_3.

[20] Fabrianne Effendi e Anupam Chattopadhyay. *Privacy-Preserving Graph-Based Machine Learning with Fully Homomorphic Encryption for Collaborative Anti-Money Laundering*. 2024. arXiv: 2411.02926 [cs.CR]. URL: https://arxiv.org/abs/2411.02926.

[21] Haibo Zhang et al. «A Review on Machine Unlearning». In: *SN Computer Science* 4.4 (apr. 2023). ISSN: 2661-8907. DOI: 10.1007/s42979-023-01767-4. URL: http://dx.doi.org/10.1007/s42979-023-01767-4.

[22] OWASP. *OWASP Machine Learning Security Top Ten 2023 | ML03:2023 Model Inversion Attack | OWASP Foundation*. [Online; accessed 2025-02-04]. 2023. URL: https://owasp.org/www-project-machine-learning-security-top-10/docs/ML03_2023-Model_Inversion_Attack.

[23] OWASP. *OWASP Machine Learning Security Top Ten 2023 | ML04:2023 Membership Inference Attack | OWASP Foundation*. [Online; accessed 2025-02-04]. 2023. URL: https://owasp.org/www-project-machine-learning-security-top-10/docs/ML04_2023-Membership_Inference_Attack.

[24] Martin Abadi et al. «Deep Learning with Differential Privacy». In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. CCS'16. ACM, ott. 2016. DOI: 10.1145/2976749.2978318. URL: http://dx.doi.org/10.1145/2976749.2978318.

[25] Nicolas Papernot et al. *Scalable Private Learning with PATE*. 2018. arXiv: 1802.08908 [stat.ML]. URL: https://arxiv.org/abs/1802.08908.

[26] Daniel Peterson, Pallika Kanani e Virendra J. Marathe. *Private Federated Learning with Domain Adaptation*. 2019. arXiv: 1912.06733 [cs.LG]. URL: https://arxiv.org/abs/1912.06733.

[27] Marco Gaboardi, Michael Hay e Salil Vadhan. «A Programming Framework for OpenDP». In: *OpenDP* (2020).

[28] Tumult Labs. *Tumult Analytics: a robust, easy-to-use, scalable, and expressive framework for differential privacy*. URL: https://cdn.prod.website-files.com/62a8b741bce9e9389c905a88/6390203b5b854c343256ab92_TumultWhitepaper.pdf.

[29] Stanley L. Warner. «Randomized Response: A Survey Technique for Eliminating Evasive Answer Bias». In: *Journal of the American Statistical Association* 60.309 (1965). PMID: 12261830, pp. 63–69. DOI: 10.1080/01621459.1965.10480775. eprint: https://www.tandfonline.com/doi/pdf/10.1080/01621459.1965.10480775. URL: https://www.tandfonline.com/doi/abs/10.1080/01621459.1965.10480775.

[30] Leonard J. Kazmier. *Schaum's easy outline of business statistics*. McGraw-Hill Trade, 2003.

[31] Google. *pbeam package*. URL: https://pkg.go.dev/github.com/google/differential-privacy/privacy-on-beam/v3/pbeam?utm_source=godoc.